



iOS SDK 1.12.0.0 for React Native User Guide



This document is intended only for authorized individuals of AppSealing customer.
Unauthorized distribution of any parts of this document to other parties is prohibited.

The software referenced herein, this User Guide, and any associated documentation is provided to you pursuant to the agreement between your company, governmental body or other entity (“you”) and Doverrunner Corporation (“AppSealing”) under which you have received a copy of AppSealing Licensed Technology and this User Guide (such agreement, the “Agreement”). Defined terms not defined herein shall have the meanings ascribed to them in the Agreement. In the event of conflict between the terms of this User Guide and the terms of the Agreement, the terms of the Agreement shall prevail. Without limiting the generality of the remainder of this paragraph, (a) this User Guide is provided to you for informational purposes only, (b) your right to access, view, use, and copy this User Guide is limited to the rights and subject to the applicable requirements and limitations set forth in the Agreement, and (c) all of the content of this User Guide constitutes “Confidential Information” of AppSealing (or the equivalent term used in the Agreement) and is subject to all of the limitations and requirements pertinent to the use, disclosure and safeguarding of such information. Permitting anyone who is not directly involved in the authorized use of AppSealing Licensed Technology by your company or other entity to gain any access to this User Guide shall violate the Agreement and subject your company or other entity to liability therefore.

Copyright Information

Copyright © 2000-2026 Doverrunner Corporation. All rights reserved.

AppSealing® is a trademark of Doverrunner Corporation in the South Korea and/or other countries.

macOS™ and Xcode® are trademarks of Apple Inc., registered in the United States and other countries.

All other trademarks are the property of their respective owners.

Disclaimer

The remainder of this User Guide notwithstanding, this User Guide is provided “as is”, without any warranty whatsoever (including that it is error-free or complete). This User Guide contains no express or implied warranties, covenants or grants of rights or licenses, and does not modify or supplement any express warranty, covenant or grant of rights or licenses that is set forth in the Agreement. This User Guide is current as of the date set forth in the header that appears above on this page, but may be modified at any time without prior notice. Future revisions and updates of this User Guide shall be distributed as part of AppSealing SDK new releases. Doverrunner shall under no circumstances bear any responsibility for your failure to operate AppSealing Licensed Technology in compliance with the then-current version of this User Guide. Your remedies with respect to your use of this User Guide, and Doverrunner's liability for your use of this User Guide (including for any errors or inaccuracies that appear in this User Guide) are limited to those remedies expressly authorized by the Agreement (if any).

Contact Information

Address: [06109] 1F 608, Nonhyeon-ro, Gangnam-gu, Seoul, South Korea

Technical support: <https://helpcenter.appsealing.com>

Website: <https://doverrunner.com>

목차

요구 사항	4
Part 1. React Native 프로젝트에 Hermes 엔진 사용 설정	
1-1 Podfile 수정	5
1-2 AppSealing SDK 압축 해제	8
1-3 프로젝트 Bundle ID 와 SDK에 등록된 Bundle ID 비교	10
Part 2. 프로젝트에 AppSealing 파일 추가	
2-1 Xcode 프로젝트에 파일 추가	12
2-2 React Native 0.70.0 이후 버전의 추가 작업	17
2-3 PrivacyInfo.xcprivacy 파일에 항목 추가 (App Store upload)	19
Part 3. 보안 위협 안내를 위한 GUI 추가	
Part 4. "Build Settings" 수정	26
Part 5. 앱 빌드 및 후처리 과정	
5-1 앱 빌드 모드에 따른 주의 사항	29
5-2 앱 무결성 및 인증서 검증 정보 생성	30
5-3 메소드 스위칭/메소드 후킹 감지 기능 제어	36
5-4 재서명 앱 App Store Connect 업로드	38
Part 6. AppSealing 기기 고유 ID 획득	
6-1 기기 고유 ID 확인	43
Part 7. 문제 해결	
7-1 Xcode Build error : Undefined symbols for architecture arm64: "_OBJC_CLASS_\$_AppSealingInterface"	44
7-2 Execution error (1) 앱이 실행되고 곧바로 종료됩니다	47
7-3 Execution error (2) 앱이 실행 중 갑자기 종료됩니다	48
7-4 Execution error (3) 앱이 실행 즉시 종료됩니다	49
7-5 무결성/인증서 스냅샷 생성 스크립트가 실행되지 않습니다	50

요구사항

- Xcode 15.0 이상 (macOS 14 이상)
- iOS 15.0 이상 기기
- React Native 0.71 이상 0.83 이하 (Release 빌드 모드만 지원)

Supported CI/CD Tool

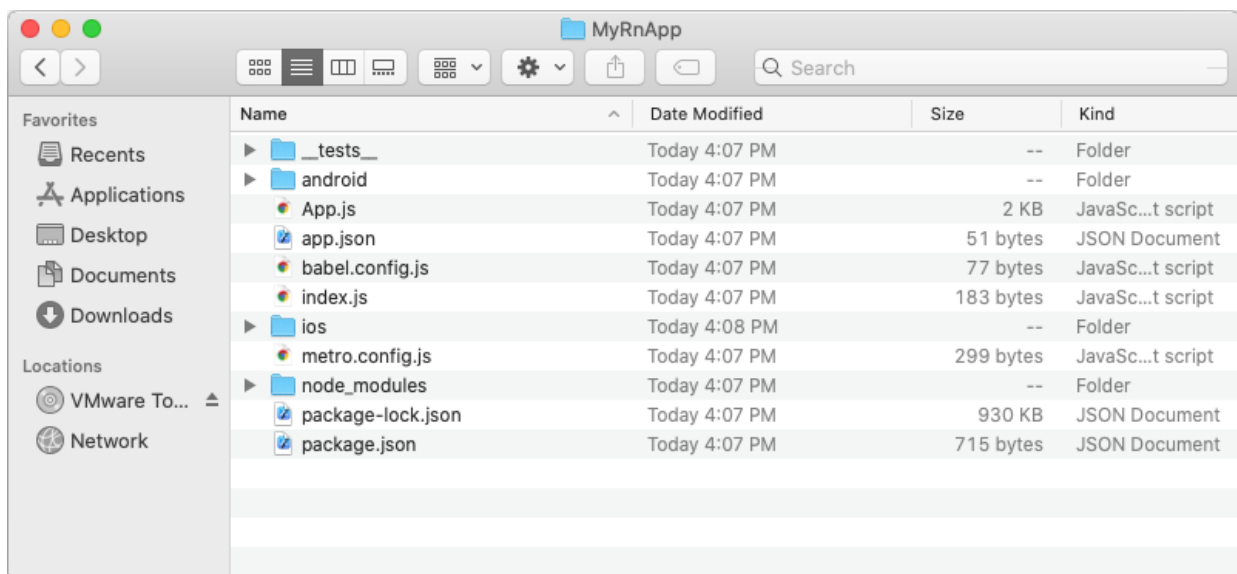
- Xcode Cloud
- Fastlane
- Bitrise
- Circle CI
- Azure

Part 1. React Native 프로젝트에 Hermes 엔진 사용 설정

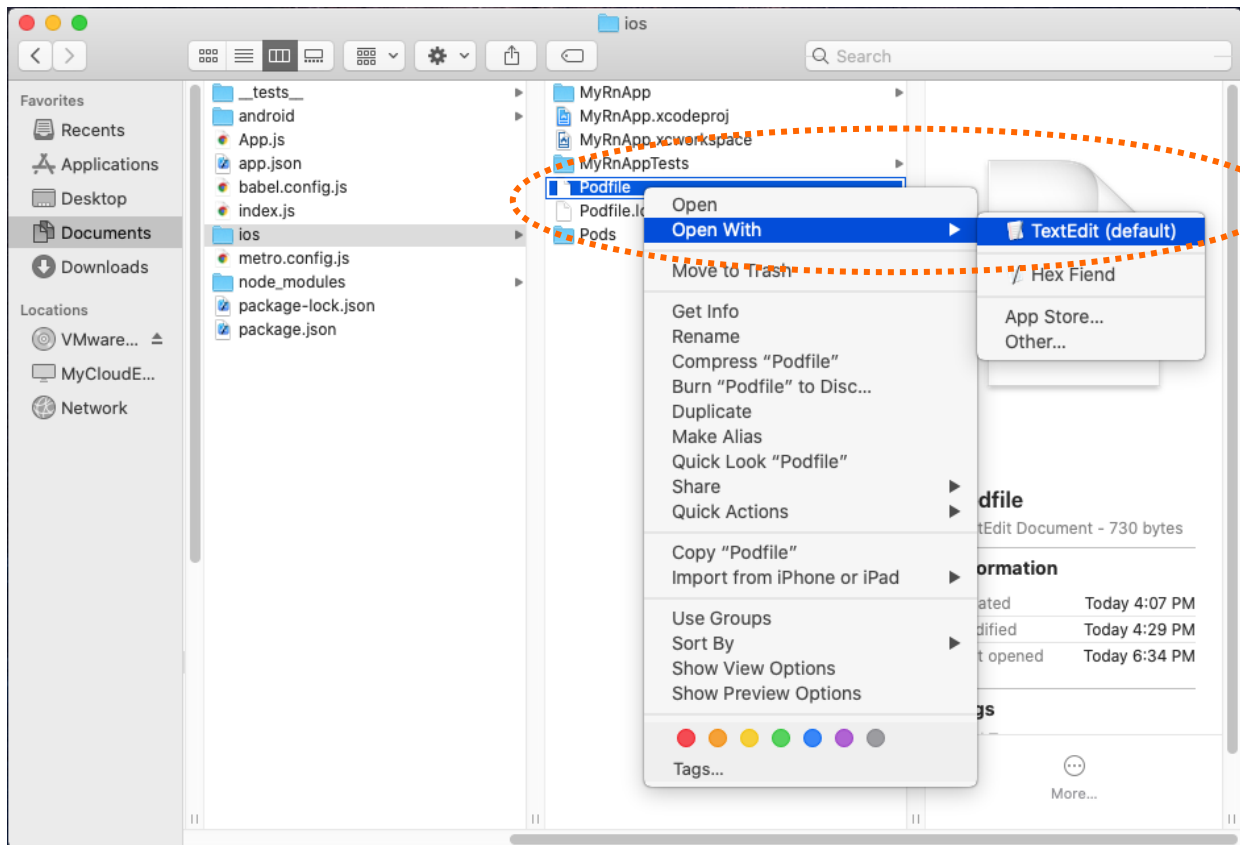
React Native 앱에 AppSealing을 적용하기 위해서는 먼저 프로젝트에 hermes 자바스크립트 엔진을 사용하도록 설정하고 AppSealing 보안 로직이 적용된 Hermes 라이브러리로 대체하여 사용해야 합니다. (AppSealing 적용은 앱 개발 마지막 단계에 진행을 권장합니다.)

1-1 Podfile 수정

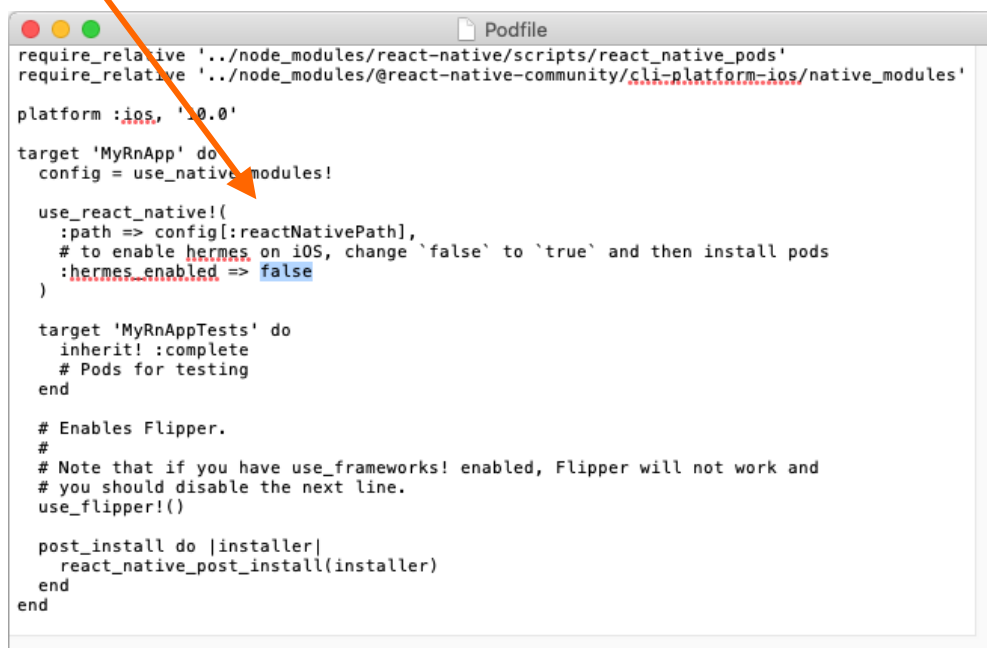
이 문서에서 사용할 React Native 프로젝트의 이름은 MyRnApp입니다. 최초 프로젝트를 생성하고 나면 다음과 같은 파일들을 확인할 수 있습니다.



이 상태에서 hermes 라이브러리를 사용하도록 ios 폴더의 Podfile 내용을 수정해야 합니다. 프로젝트 폴더에서 ios 폴더를 선택하고 그 안에 있는 Podfile을 마우스 오른쪽 버튼으로 클릭하여 "Open With"를 선택하고 "TextEdit (default)"를 선택합니다.

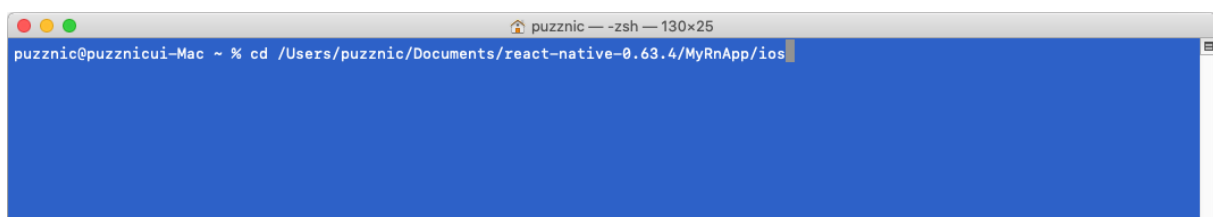


파일이 열리면 ':hermes_enabled => false' 라고 되어 있는 라인을 찾아 false를 true로 바꿔 줍니다. (버전 0.73.x 이상에서는 해당 라인이 없을 경우 조치하지 않으셔도 됩니다.)

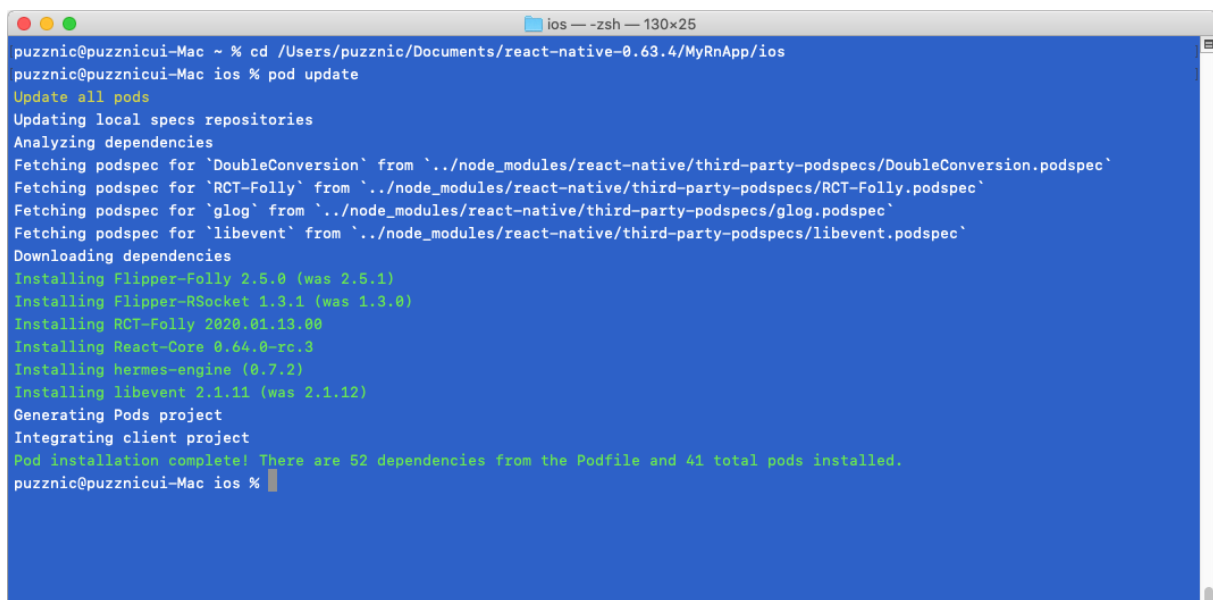




수정이 완료되면 터미널을 열고 MyRnApp/ios 폴더로 이동합니다.

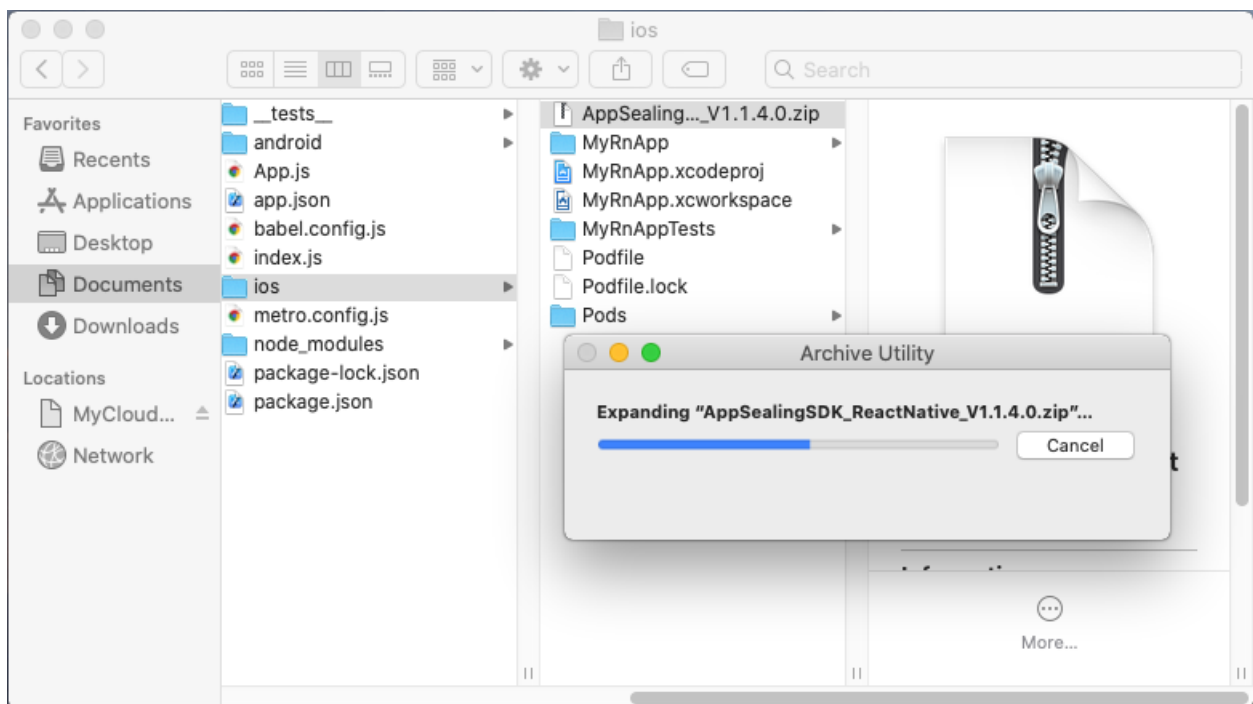
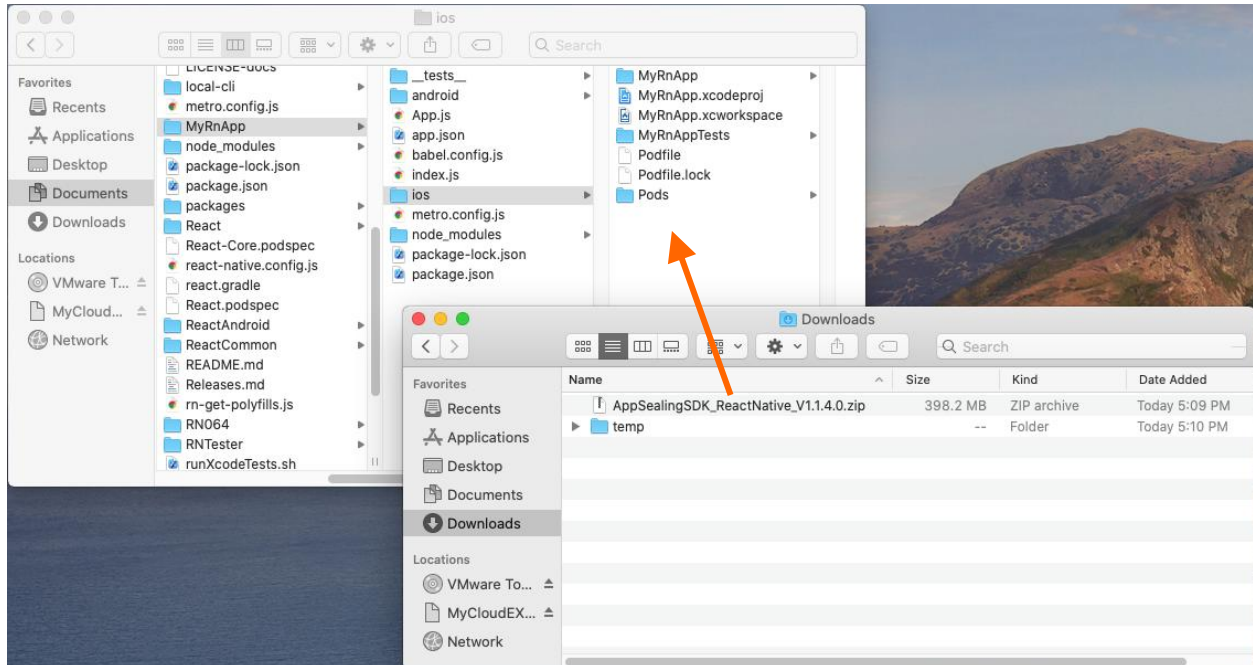


ios 폴더에서 pod update 명령을 실행하여 hermes 엔진을 사용하기 위해 필요한 추가 구성 요소들이 설치되도록 합니다.

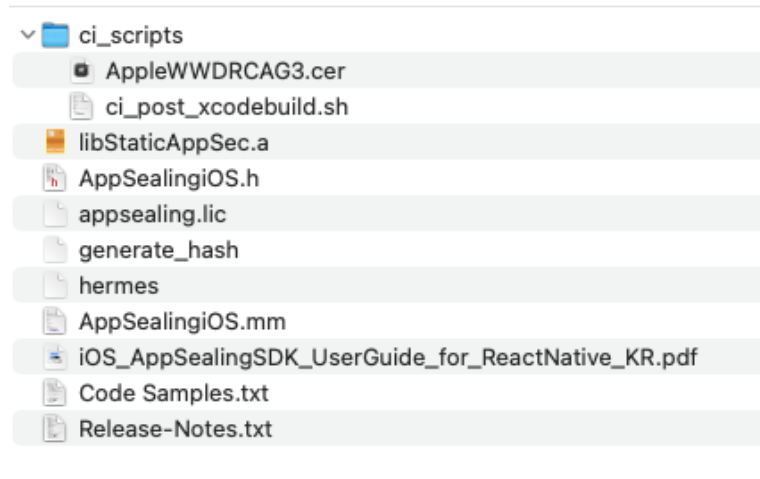


1-2 AppSealing SDK 압축 해제

ADC에서 내려 받은 AppSealing_SDK_ReactNative.zip 파일을 ios 폴더로 이동시키고 더블 클릭하여 압축을 해제합니다.



Zip 파일 압축 해제 후 다음과 같은 파일들이 모두 포함되었는지 확인하십시오.

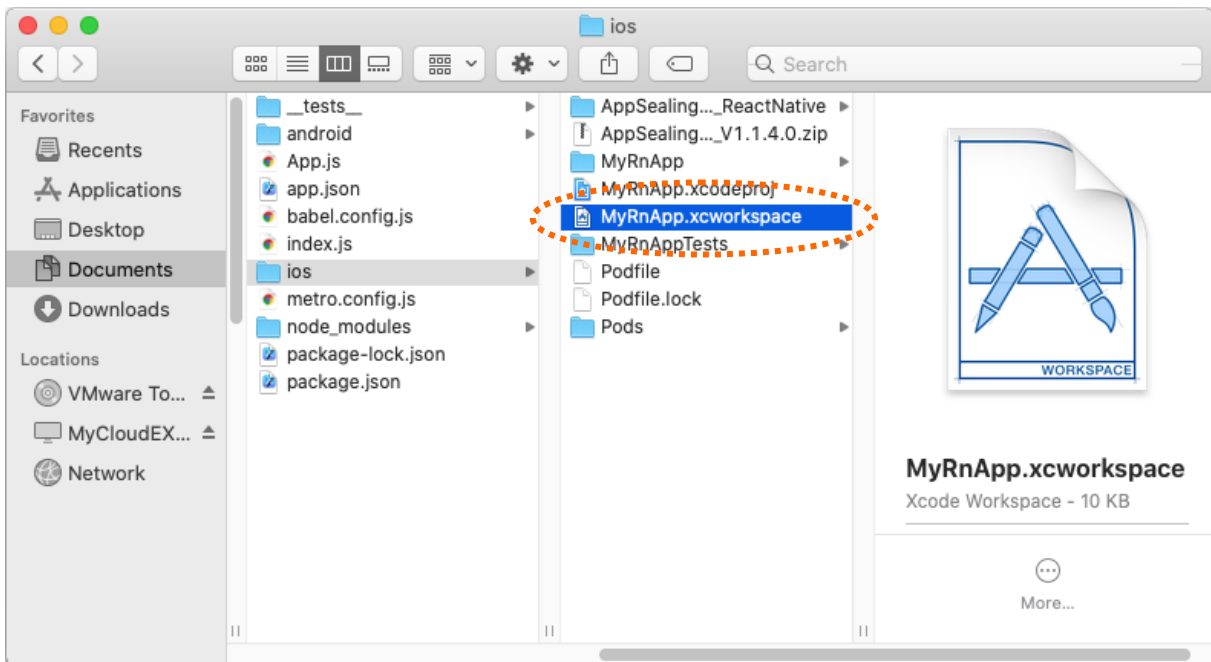


파일이 없는 경우 SDK를 다시 내려 받거나 zip 파일 압축을 다시 해제해 보십시오.

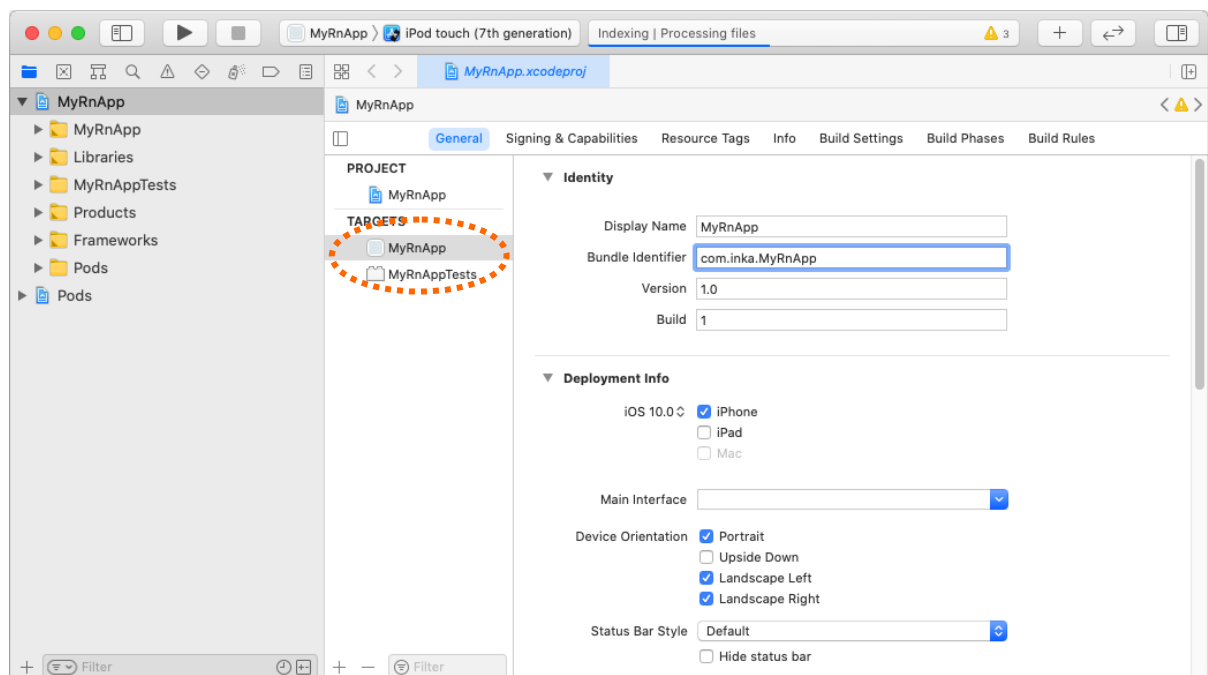
**** 주의: SDK 압축 파일의 이름이 "AdhocEnabled"를 포함하는 버전은 공식 App Store 혹은 TestFlight 이외의 방식(Ad Hoc, enterprise, MDM 및 기타 third party store)으로 앱을 배포할 수 있도록 일부 보안 기능이 약화된 SDK입니다. App Store을 통하지 않은 배포는 공격에 취약해질 수 있으므로, 반드시 필요한 경우가 아니라면 AdhocEnabled가 아닌 버전으로 다운로드 받으시기 바랍니다.**

1-3 프로젝트 Bundle ID 와 SDK에 등록된 **Bundle ID** 비교

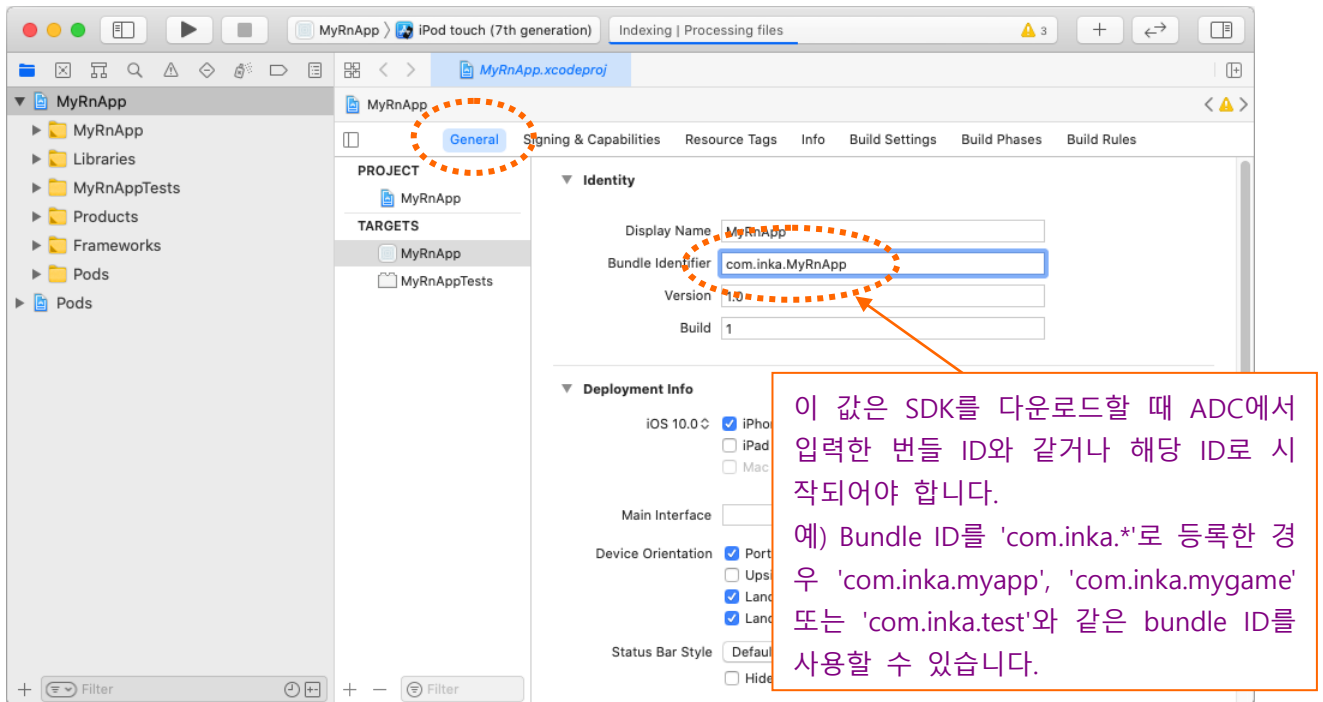
SDK zip 파일의 압축을 해제한 뒤 프로젝트의 bundle ID가 유효한지 확인하십시오.
프로젝트의 ios 폴더에서 MyRnApp.xcworkspace 파일을 더블 클릭하여 Xcode에서 프로젝트를 엽니다.



Xcode의 왼쪽 패널에서 프로젝트를 선택하고 Targets 목록에서 프로젝트 이름 (MyRnApp)을 선택하십시오.



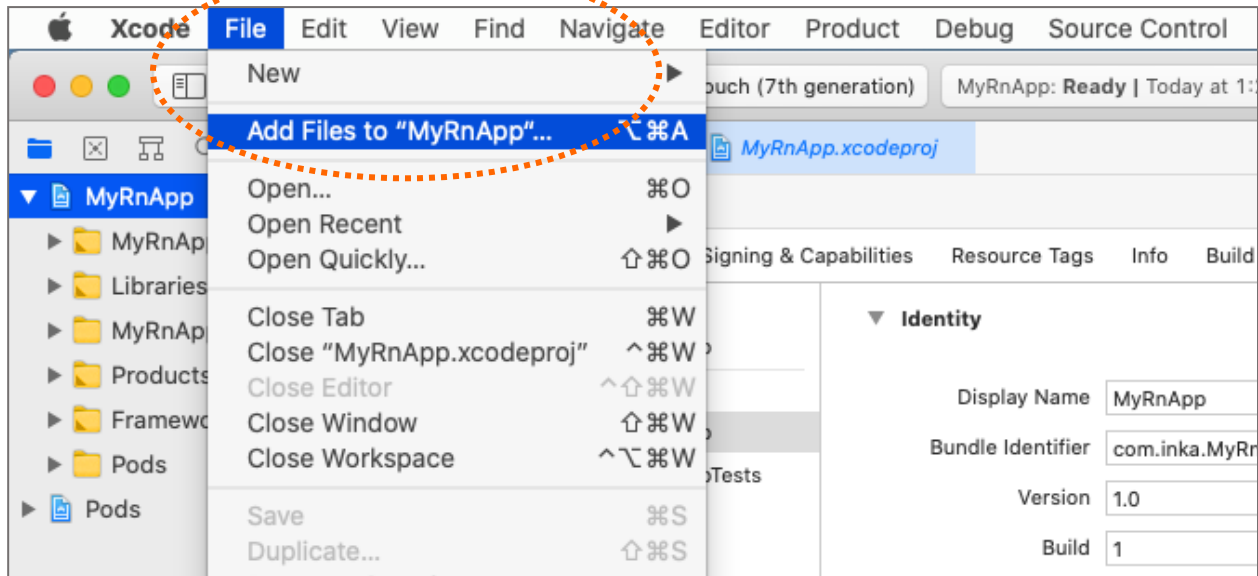
그 다음 "General" 탭을 선택하고 bundle ID가 AppSealing Developer Center (ADC)에서 SDK를 다운로드할 때 입력한 값과 같거나 해당 값으로 시작되는지 확인하십시오.



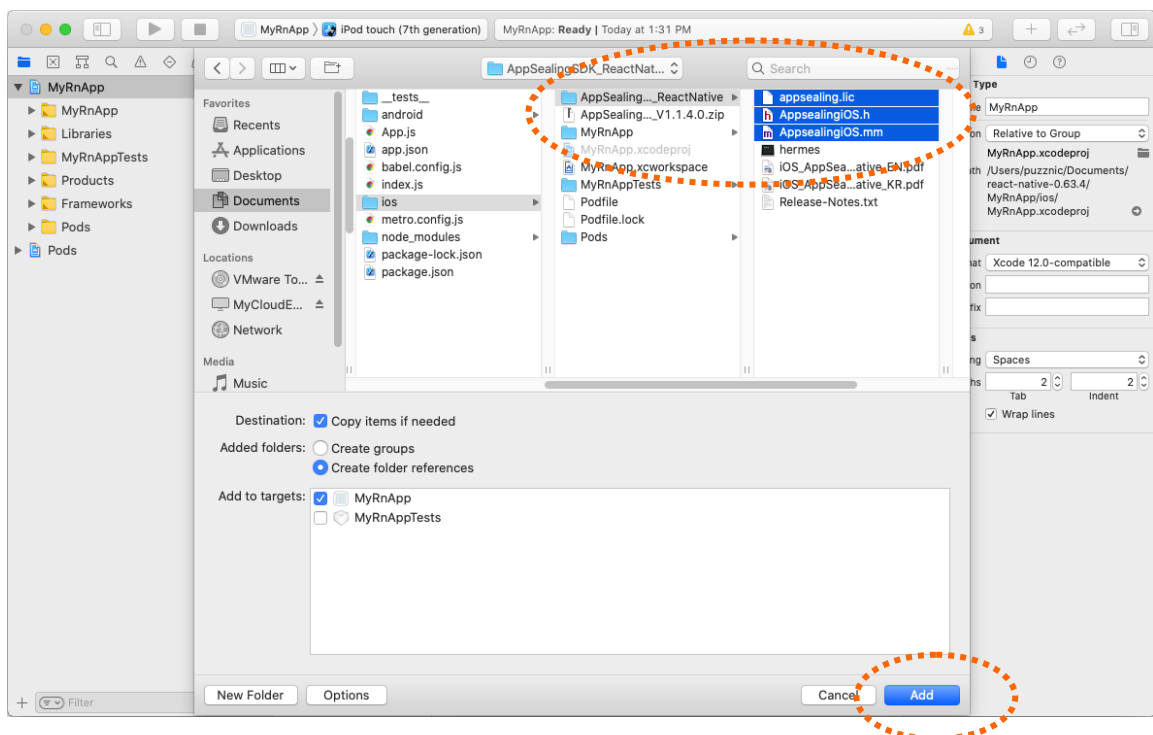
Part 2. 프로젝트에 AppSealing 파일 추가

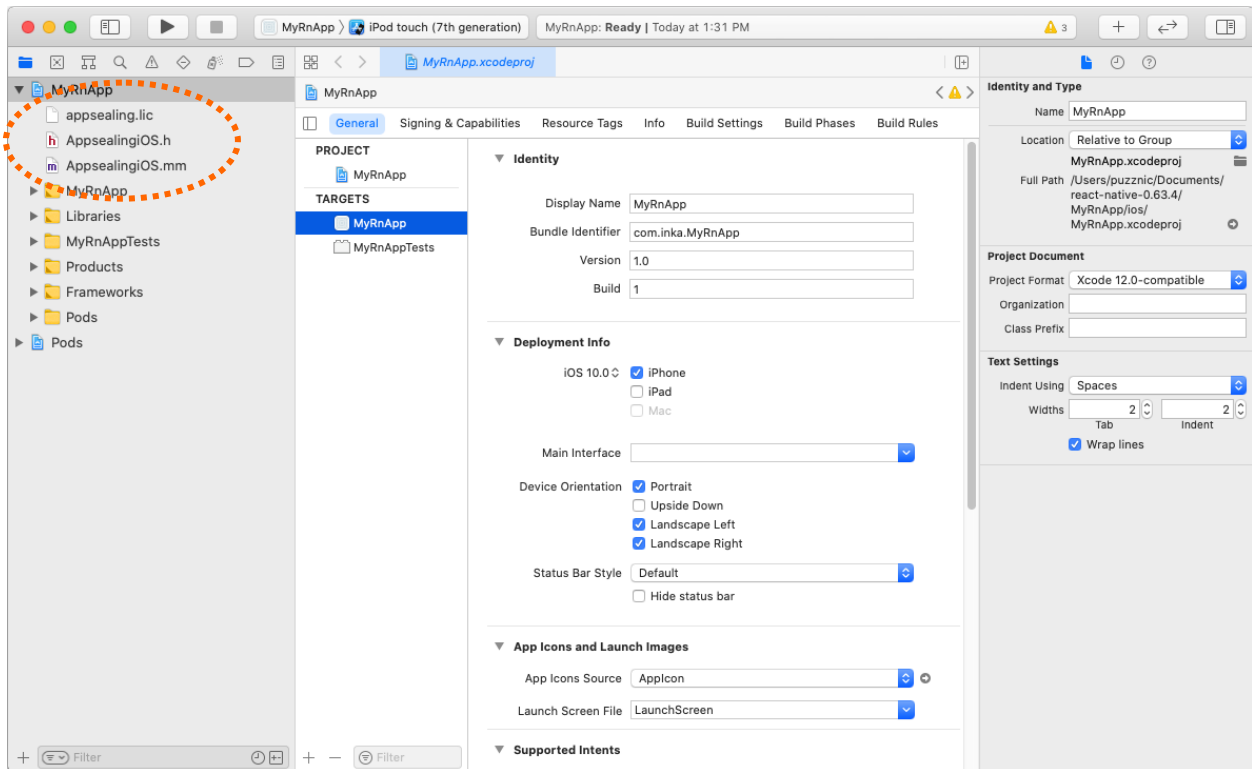
2-1 Xcode 프로젝트에 파일 추가

이제 Xcode 프로젝트에 필요한 파일들을 추가해야 합니다. 먼저 상단 메뉴에서 "File" 항목을 선택하고 하위 메뉴 중 "Add Files to "MyRnApp:..." 항목을 선택합니다.

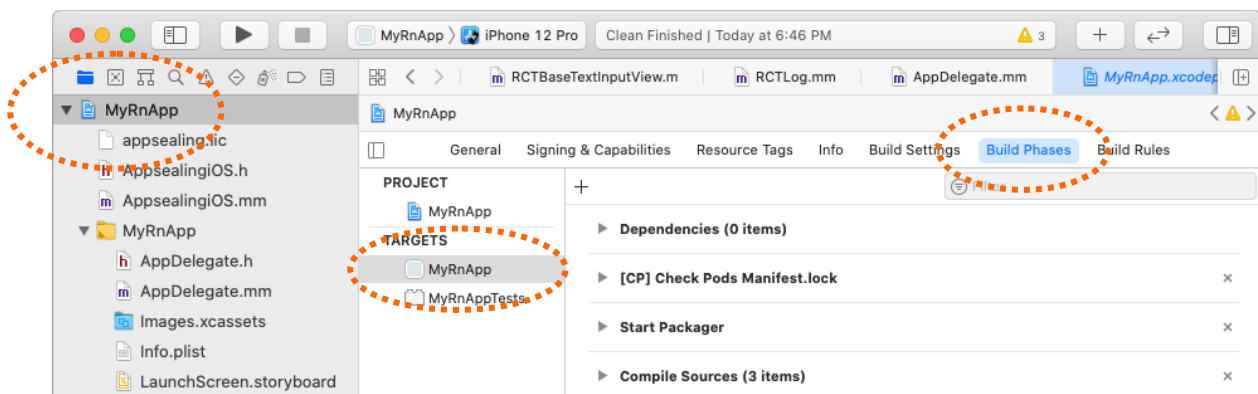


"AppSealingSDK_ReactNative/" 폴더를 선택한 후 **"appsealing.lic"**, **"AppsealingiOS.h"**, **"AppsealingiOS.mm"** 파일을 선택하고 "Add" 버튼을 눌러 프로젝트에 추가하십시오.

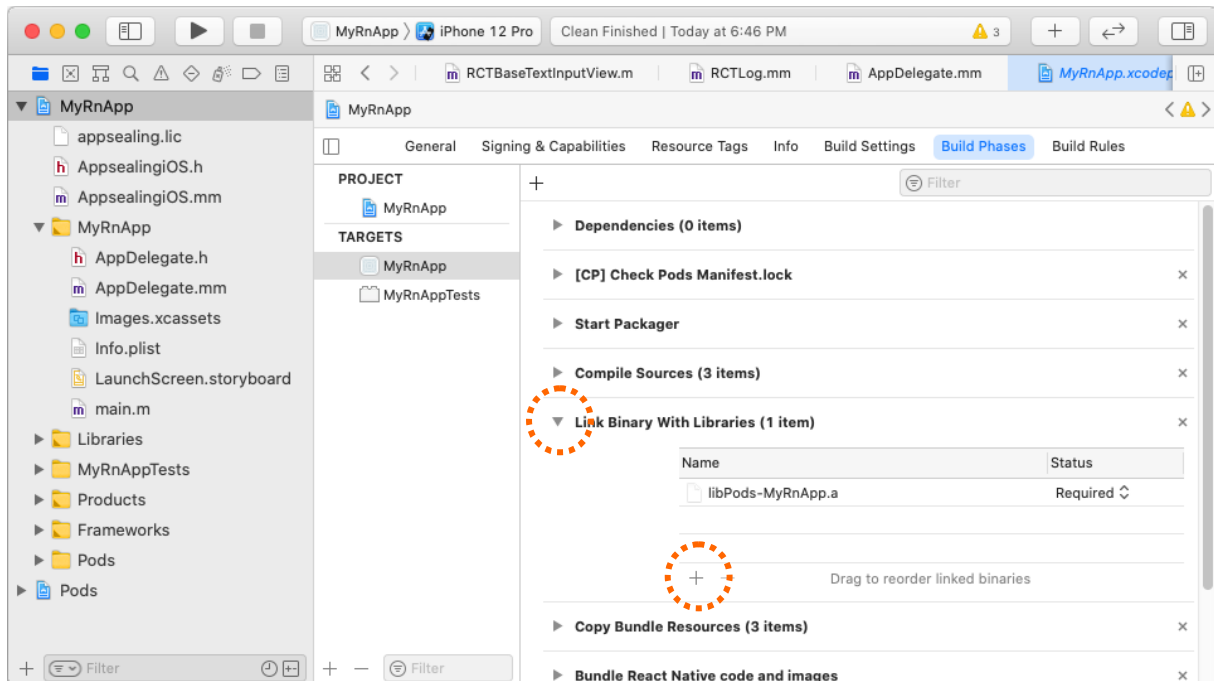




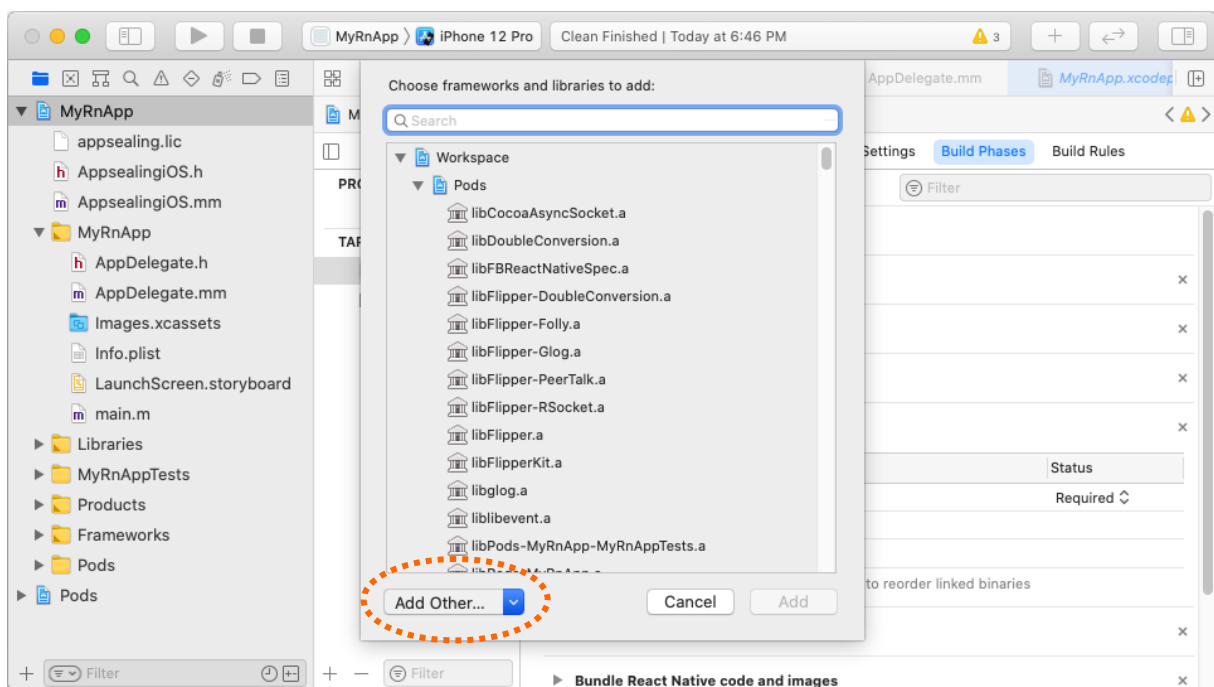
다음 단계는 AppSealing 정적 라이브러리를 프로젝트에 추가하는 단계입니다. TARGETS에서 프로젝트 명이 선택되어 있는 상태에서 오른쪽 탭의 "Build Phases"를 선택합니다.



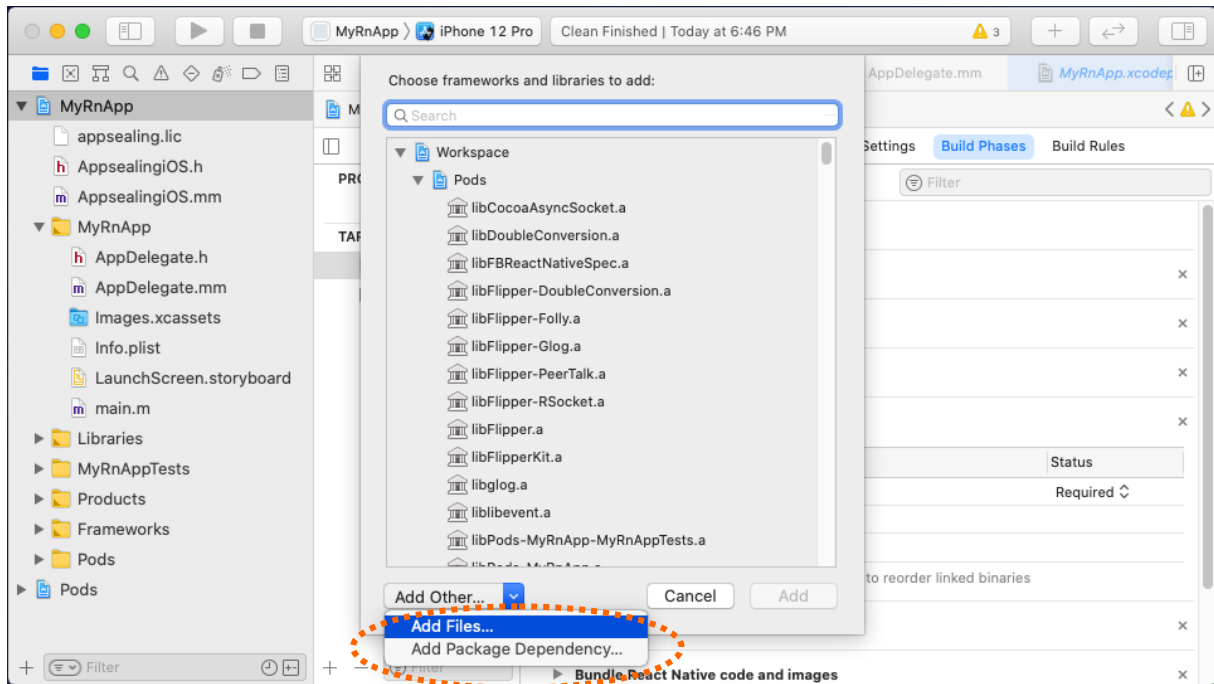
“Link Binary With Libraries” 왼쪽의 화살표 아이콘을 클릭하여 옵션을 확장한 뒤 아래쪽의 + 아이콘을 클릭합니다.



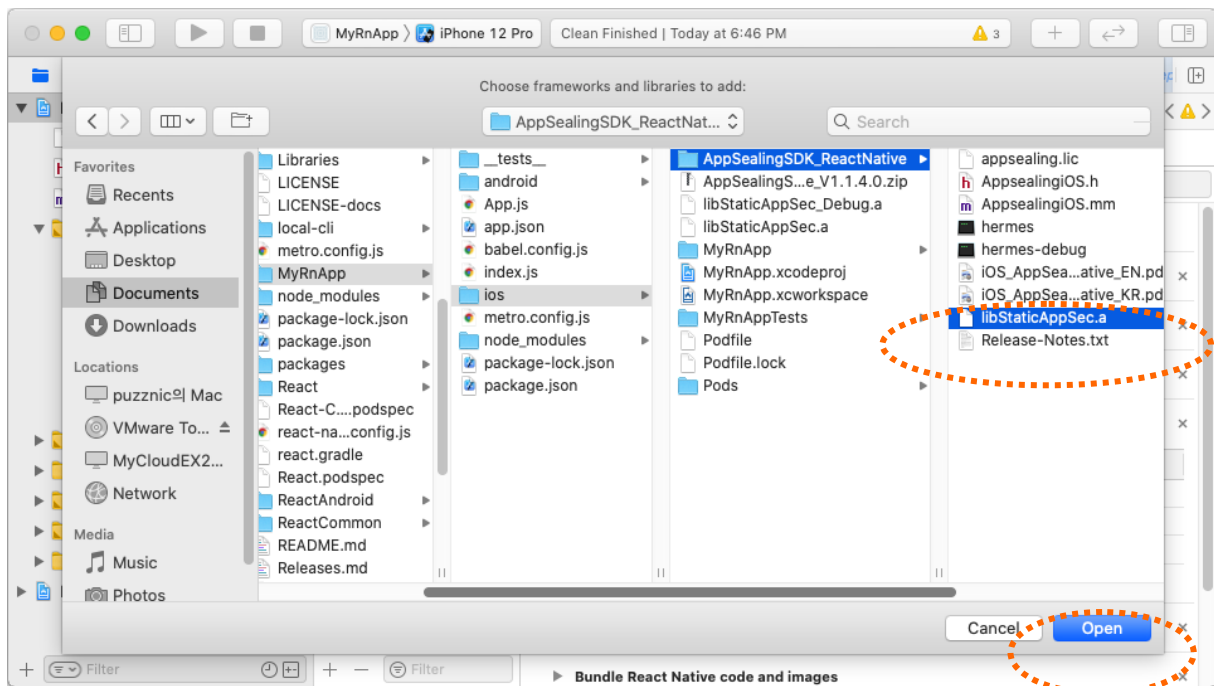
파일 선택 창이 나타나면 하단의 “Add Other...” 버튼을 클릭합니다.



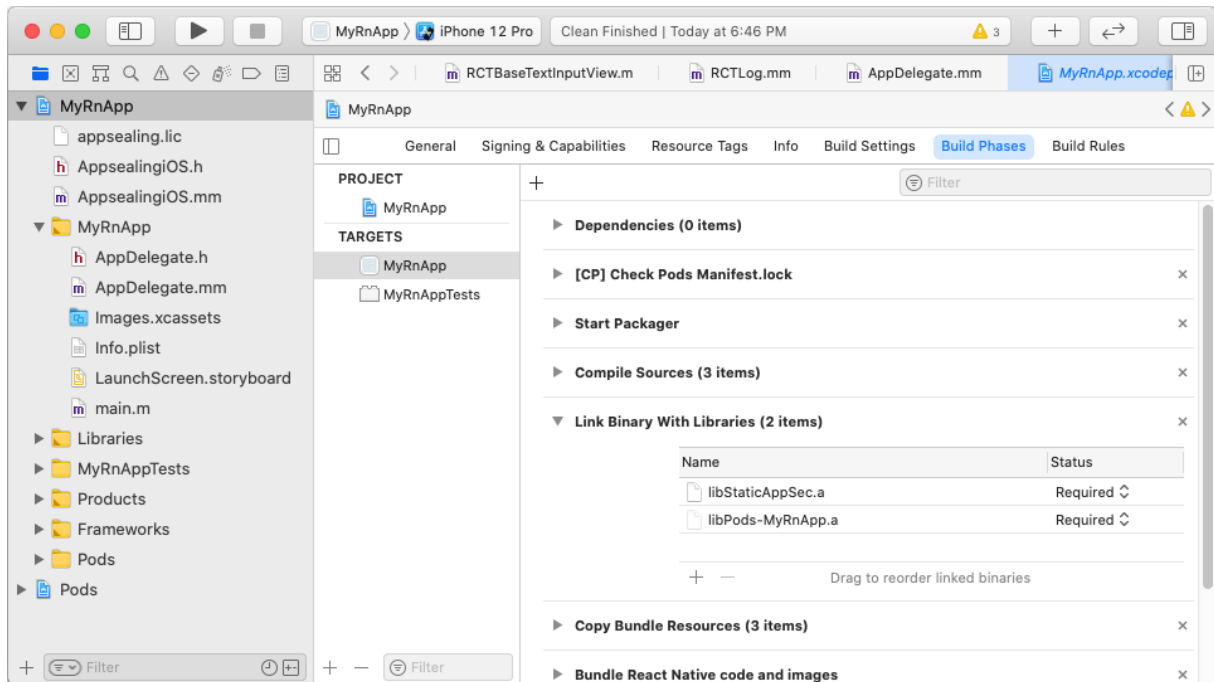
드롭 다운 목록에서 "Add Files..." 항목을 선택합니다.



파일 오픈 대화 상자가 표시되면 MyRnApp/ios/AppSealingSD_ReactNative 폴더의 libStaticAppSec.a 파일을 선택하고 "Open" 버튼을 클릭합니다.



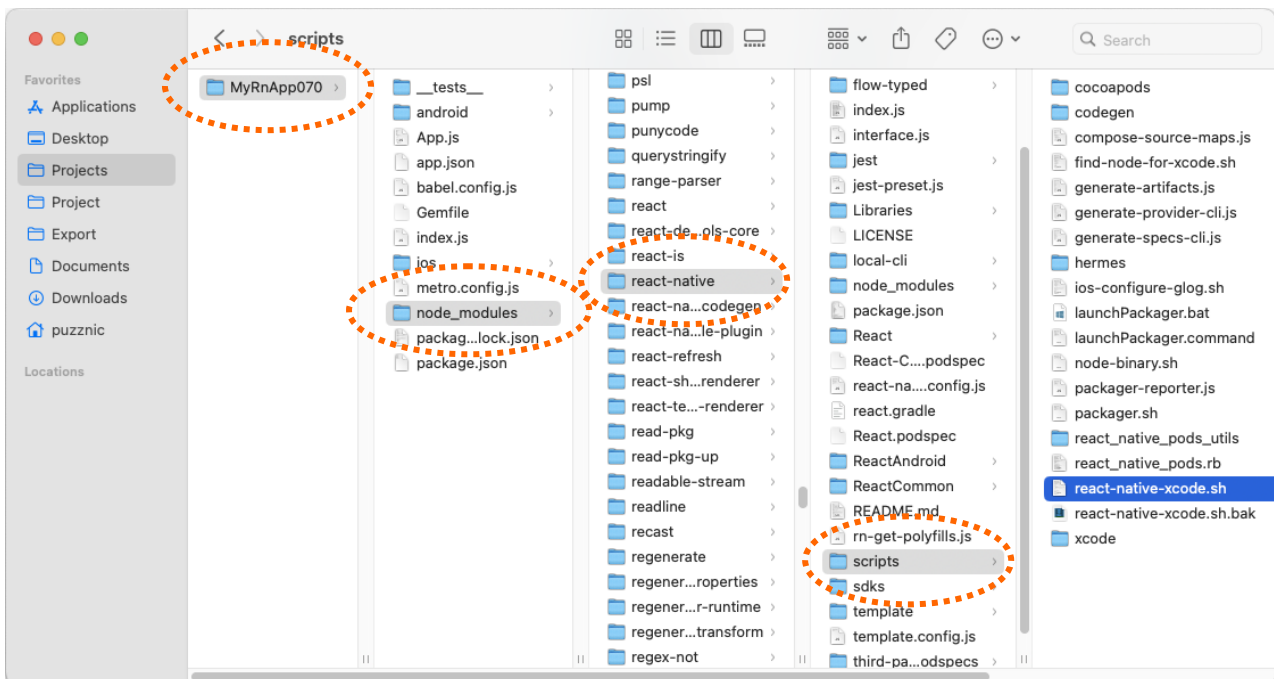
아래 화면처럼 libStaticAppSec.a 항목이 추가된 것을 확인할 수 있습니다.



2-2 React Native 0.70.0 이후 버전의 추가 작업

AppSealing SDK를 적용할 React Native 앱의 버전이 0.70.0 이상이라면 다음의 추가 작업을 진행해 주어야 합니다. **이 과정이 생략될 경우 자바스크립트 코드가 정상적으로 암호화 되지 않습니다.**

먼저 Finder를 열고 해당 프로젝트 경로의 `node_modules/react-native/scripts` 폴더로 이동합니다.



폴더로 이동한 후 `react-native-xcode.sh` 라는 파일을 찾아 TextEditor 앱으로 엽니다. 파일을 오픈하면 스크립트 코드의 84 라인으로 이동하여 아래 스크립트를 추가해 줍니다.

```
if [[ -z "$USE_HERMES" && -f "$HERMES_CLI_PATH" ]]; then
    echo "Enabling Hermes byte-code compilation. Disable with USE_HERMES=false if
    needed."
    USE_HERMES=true
fi
```

```

react-native-xcode.sh — Edited
react-native-xcode.sh No Selection
62 # in node_modules.
63 PROJECT_ROOT=${PROJECT_ROOT:-"$REACT_NATIVE_DIR/../../"}
64
65 cd "$PROJECT_ROOT" || exit
66
67 # Define entry file
68 if [[ "$ENTRY_FILE" ]]; then
69   # Use ENTRY_FILE defined by user
70   :
71 elif [[ -s "index.ios.js" ]]; then
72   ENTRY_FILE=${1:-index.ios.js}
73 else
74   ENTRY_FILE=${1:-index.js}
75 fi
76
77 # check and assign NODE_BINARY env
78 # shellcheck source=/dev/null
79 source "$REACT_NATIVE_DIR/scripts/node-binary.sh"
80
81 HERMES_ENGINE_PATH="$PODS_ROOT/hermes-engine"
82 [ -z "$HERMES_CLI_PATH" ] &&
83   HERMES_CLI_PATH="$HERMES_ENGINE_PATH/destroot/bin/hermes"
84
85 # Hermes is enabled in new projects by default, so we cannot assume that
86   USE_HERMES=1 is set as an envvar.
87
88 # If hermes-engine is found in Pods, we can assume Hermes has not been disabled.
89 # If hermes is not available and USE_HERMES is either unset or true, show error.
90 if [[ -f "$HERMES_ENGINE_PATH" && ! -f "$HERMES_CLI_PATH" ]]; then
91   echo "error: Hermes is enabled but the hermes binary could not be found at
92     ${HERMES_CLI_PATH}." \
93     "Perhaps you need to run 'bundle exec pod install' or otherwise " \
94     "point the HERMES_CLI_PATH variable to your custom location." >&2
95   exit 2
96 fi
97
98 [ -z "$NODE_ARGS" ] && export NODE_ARGS=""
99

```

```

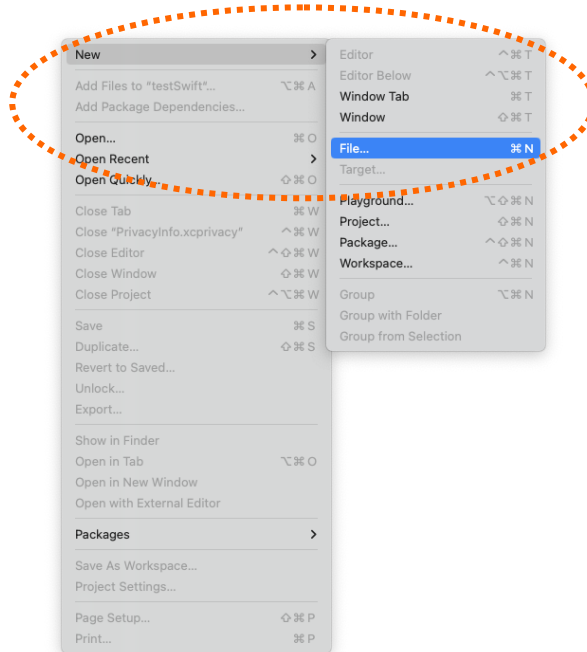
80 HERMES_ENGINE_PATH="$PODS_ROOT/hermes-engine"
81 [ -z "$HERMES_CLI_PATH" ] &&
82   HERMES_CLI_PATH="$HERMES_ENGINE_PATH/destroot/bin/hermes"
83
84 if [[ -z "$USE_HERMES" && -f "$HERMES_CLI_PATH" ]]; then
85   echo "Enabling Hermes byte-code compilation. Disable with USE_HERMES=false if
86     needed."
87   USE_HERMES=true
88 fi
89
90 # Hermes is enabled in new projects by default, so we cannot assume that
91   USE_HERMES=1 is set as an envvar.
92
93 # If hermes-engine is found in Pods, we can assume Hermes has not been disabled.
94 # If hermes is not available and USE_HERMES is either unset or true, show error.
95

```

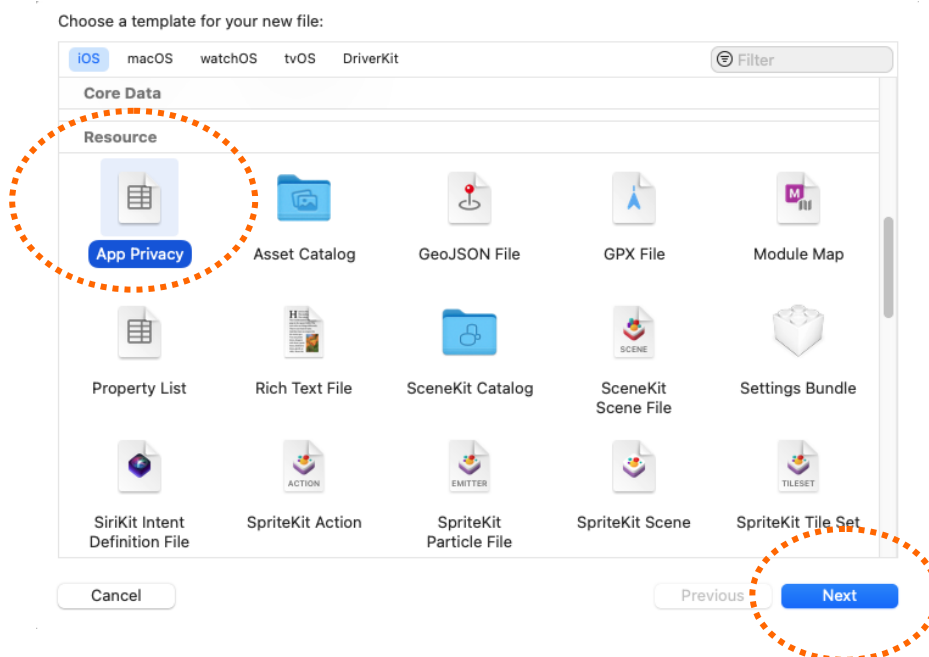
2-3 PrivacyInfo.xcprivacy 파일에 항목 추가 (App Store upload)

2024년 5월부터 App Store 업로드에 요구되는 privacy manifest 파일에도 항목 추가가 필요합니다.

현재 프로젝트에 "PrivacyInfo.xcprivacy" 파일이 없다면 프로젝트의 .xcodeproj 혹은 .xcworkspace 파일을 Xcode로 열고 메뉴의 "File > New > File..."을 선택하십시오:



"Resource" 섹션의 "App Privacy"를 선택하여 프로젝트에 privacy manifest 파일을 추가할 수 있습니다:



Privacy manifest 파일에 대한 더 자세한 설명은 아래의 링크를 참고해주시오:

https://developer.apple.com/documentation/bundleresources/privacy_manifest_files#4284009

"PrivacyInfo.xcprivacy" 파일을 열고 다음과 같은 하위 구조를 포함하도록 항목을 추가해주시오.

- key "NSPrivacyAccessedAPITypes"
 - array
 - ◆ key "NSPrivacyAccessedAPIType": string "NSCategoryFileTimestamp"
 - ◆ key "NSPrivacyAccessedAPITypeReasons"
 - array
 - string "C617.1"

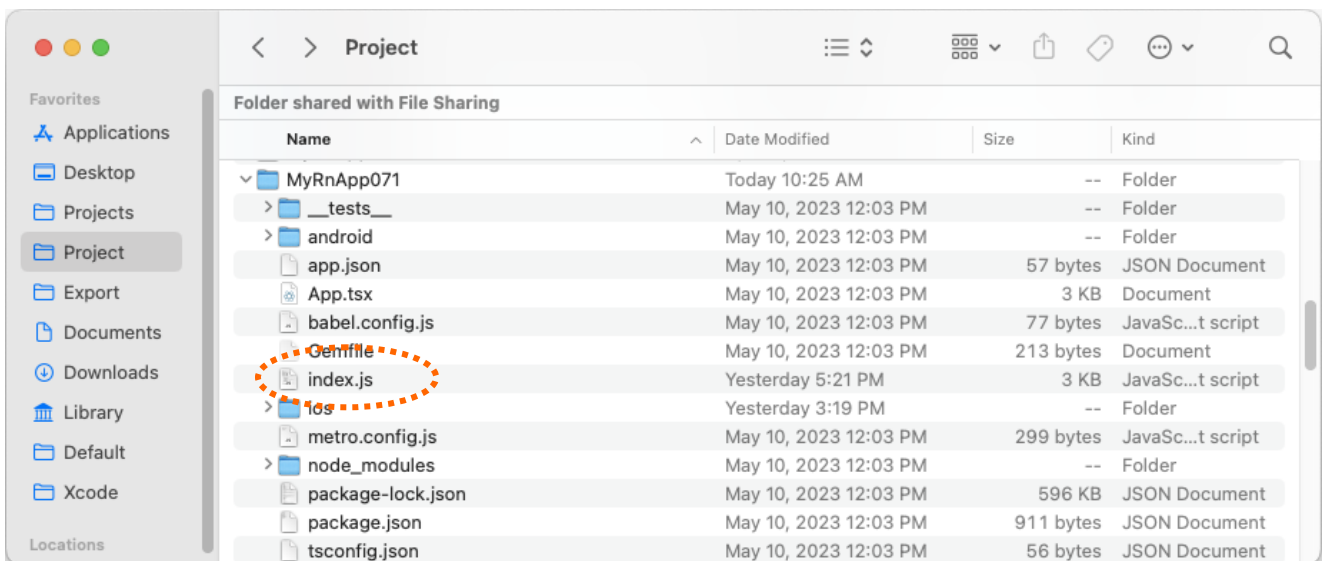
결과적으로 Xcode에서 "PrivacyInfo.xcprivacy"를 확인했을 때 다음과 같은 구조를 포함해야 합니다:

▼ Privacy Accessed API Types	↕	Array	(1 item)	
▼ Item 0		Dictionary	(2 items)	
Privacy Accessed API Type	↕	String	File Timestamp	↕
▼ Privacy Accessed API Reasons	↕	Array	(1 item)	
Item 0		String	C617.1: Inside app or group container, per documentation	↕

Part 3. 보안 위협 안내를 위한 GUI

AppSealing 라이브러리는 앱이 실행될 때 자동으로 활성화됩니다. AppSealing 라이브러리가 비정상적인 환경 (탈옥된 장치, 실행 파일의 복호화 된 상태, 디버거 연결)을 감지한 경우 사용자 조치와 관계없이 20초 후에 앱이 닫히도록 설계되어 있습니다. 앱 탐지 결과를 사용자에게 통지하고 사용자가 일부 잘못된 환경에 있음을 인식할 수 있도록 적절한 메시지 상자를 표시하십시오.

앱에서 해당 대화 상자를 표시하려면 Finder 앱을 열어 프로젝트 폴더로 이동한 후 "index.js" 파일을 선택하고 문서 편집기로 해당 파일을 열어둡니다.



편집기에서 파일을 연 후 "index.js" 파일의 가장 아래 쪽에 다음 코드를 추가합니다.

Simple UI code into 'index.js' for React Native project

```

//##### AppSealing Code-Part BEGIN: DO NOT MODIFY THIS LINE !!!
import {Alert, NativeModules} from 'react-native';

const DETECTED_JAILBROKEN = 0x00000001;
const DETECTED_DRM_DECRYPTED = 0x00000002;
const DETECTED_DEBUG_ATTACHED = 0x00000004;
const DETECTED_HASH_INFO_CORRUPTED = 0x00000008;
const DETECTED_CODESIGN_CORRUPTED = 0x00000010;
const DETECTED_HASH_MODIFIED = 0x00000020;
const DETECTED_EXECUTABLE_CORRUPTED = 0x00000040;
const DETECTED_CERTIFICATE_CHANGED = 0x00000080;
const DETECTED_BLACKLIST_CORRUPTED = 0x00000100;
const DETECTED_CHEAT_TOOL = 0x00000200;

try
{
  const {AppSealingInterfaceBridge} = NativeModules;

  let security_threat = parseInt( AppSealingInterfaceBridge.IsAbnormalEnvironmentDetectedRN() );
  if ( security_threat > 0 )
  {
    // Show GUI
    let msg = "\n-----"
      + "\nAbnormal Environment Detected !!"
      + "\n-----\n";
    if (( security_threat & DETECTED_JAILBROKEN ) > 0 )
      msg += "\n - Jailbroken";
    if (( security_threat & DETECTED_DRM_DECRYPTED ) > 0 )
      msg += "\n - Executable is not encrypted";
  }
}

```

```

        if ( ( security_threat & DETECTED_HASH_INFO_CORRUPTED ) > 0 || ( security_threat & DETECTED_HASH_MODIFIED ) > 0 )
            msg += "\n - App integrity has broken";
        if ( ( security_threat & DETECTED_CODESIGN_CORRUPTED ) > 0 || ( security_threat & DETECTED_EXECUTABLE_CORRUPTED ) > 0 )
            msg += "\n - App executable has corrupted";
        if ( ( security_threat & DETECTED_CERTIFICATE_CHANGED ) > 0 )
            msg += "\n - App has re-signed";
        if ( ( security_threat & DETECTED_BLACKLIST_CORRUPTED ) > 0 )
            msg += "\n - Blacklist/whitelist has corrupted";
        if ( ( security_threat & DETECTED_CHEAT_TOOL ) > 0 )
            msg += "\n - Cheat tool has detected";
        console.log( "AppSealing Security Threat: " + msg );
        Alert.alert( "AppSealing Security Threat", msg,
            [{ text: "Confirm", onPress: () => { AppSealingInterfaceBridge.ExitApp(); } }], { cancelable: false } );
    }
}
catch( e )
{
    console.log( "### AppSealing Security Check : " + e.ToString() );
}

AppSealingPeriodicAlerted = false;
function AppSealingPeriodicCheck() {
    if( AppSealingPeriodicAlerted ){
        return;
    }
    try
    {
        const {AppSealingInterfaceBridge} = NativeModules;

        let ret = parseInt( AppSealingInterfaceBridge.IsSwizzlingDetectedReturnRN() );

        if ( ret >= 1 && ret <= 3 )
        {
            // Show GUI
            let msg = "\n-----"
                + "\nAbnormal Environment Detected !!"
                + "\n-----\n";

            if ( ret == 1 )
                msg += "\n - Jailbroken";
            if ( ret == 2 )
                msg += "\n - Method Swizzling";
            if ( ret == 3 )
                msg += "\n - Method Hooking";

            console.log( "AppSealing Security Threat: " + msg );
            Alert.alert( "AppSealing Security Threat", msg, [{ text: "Confirm", onPress: () => { AppSealingInterfaceBridge.ExitApp(); } }],
            { cancelable: false } );

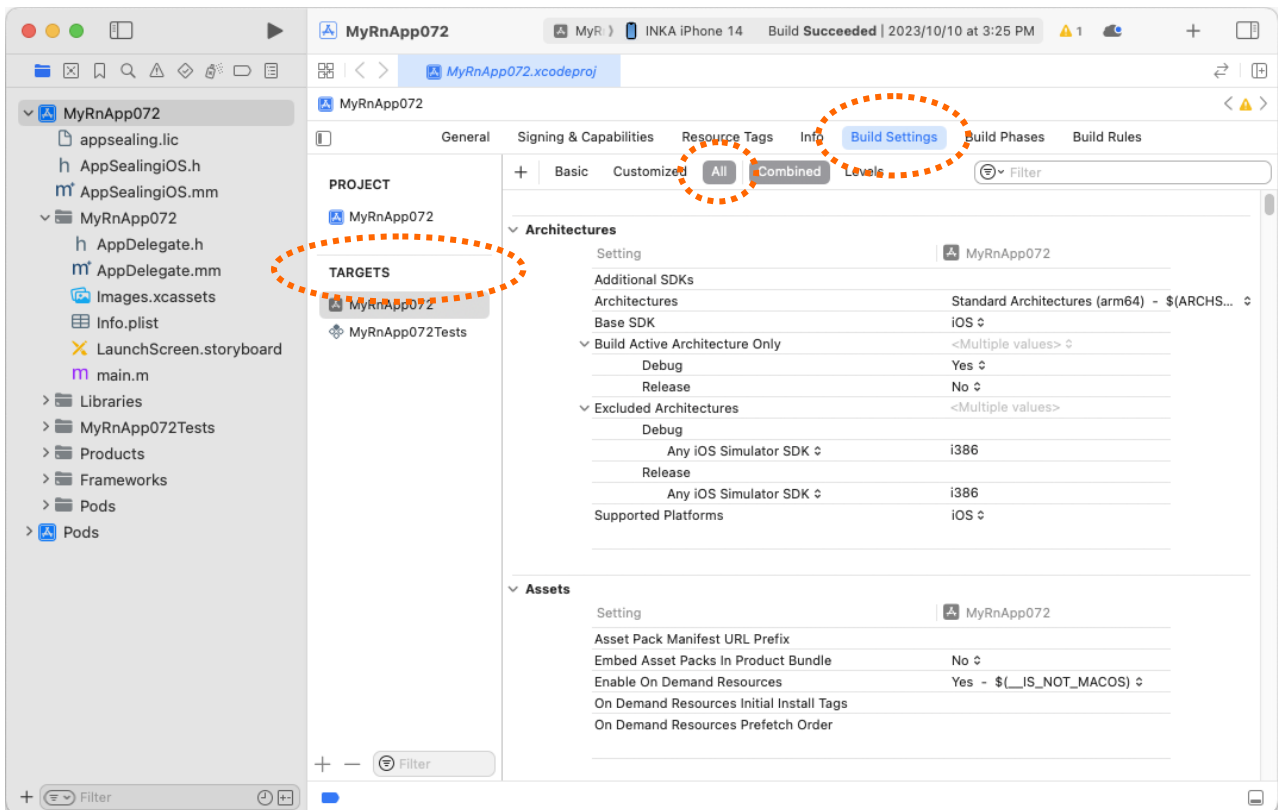
            AppSealingPeriodicAlerted = true;
        }
    }
    catch( e )
    {
        console.log( "### AppSealing Security Check : " + e.ToString() );
    }
}

setInterval(AppSealingPeriodicCheck, 2500); // 2.5ms
//##### AppSealing Code-Part END: DO NOT MODIFY THIS LINE !!!

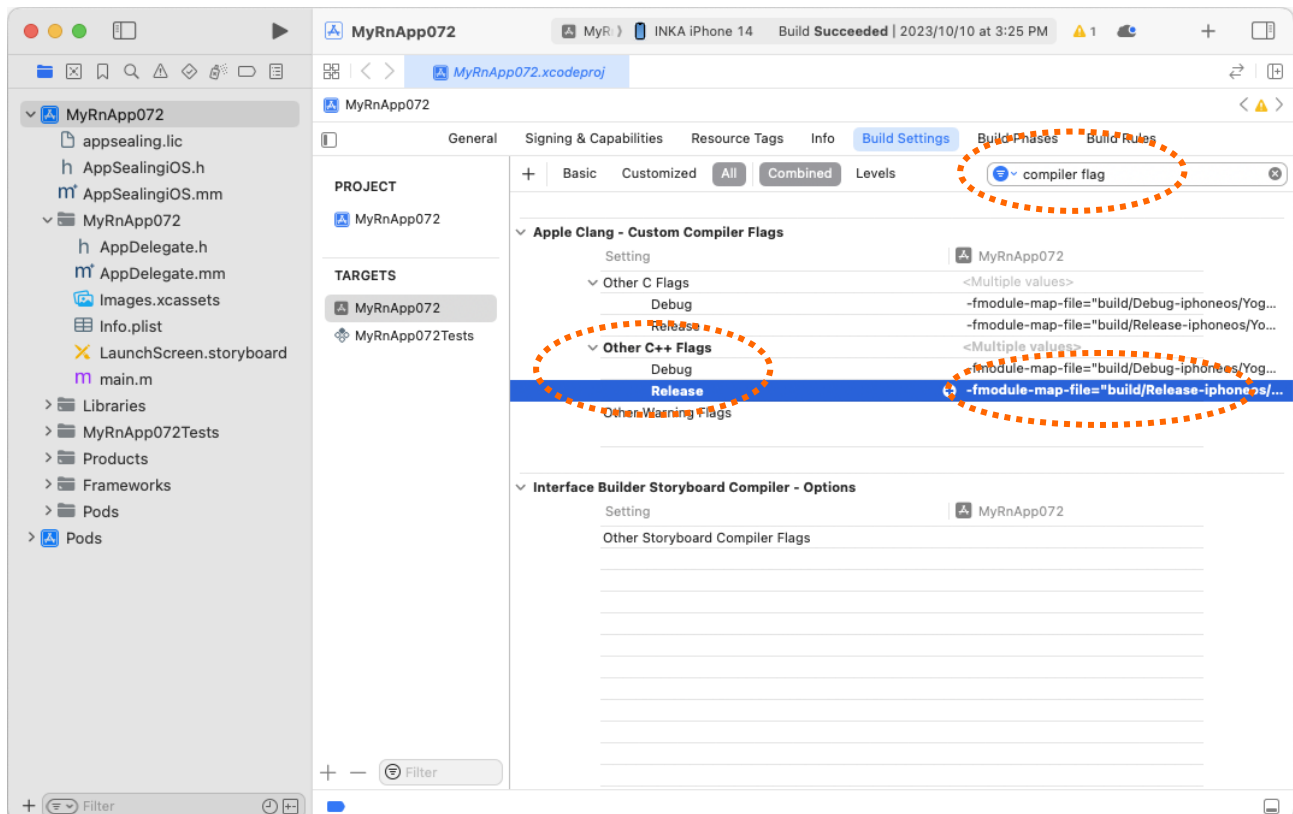
```

위 예제 코드는 AppSealing SDK에 포함된 "Code Samples.txt" 파일에도 포함되어 있으므로 그대로 복사/붙여넣기 하여 사용할 수 있습니다.

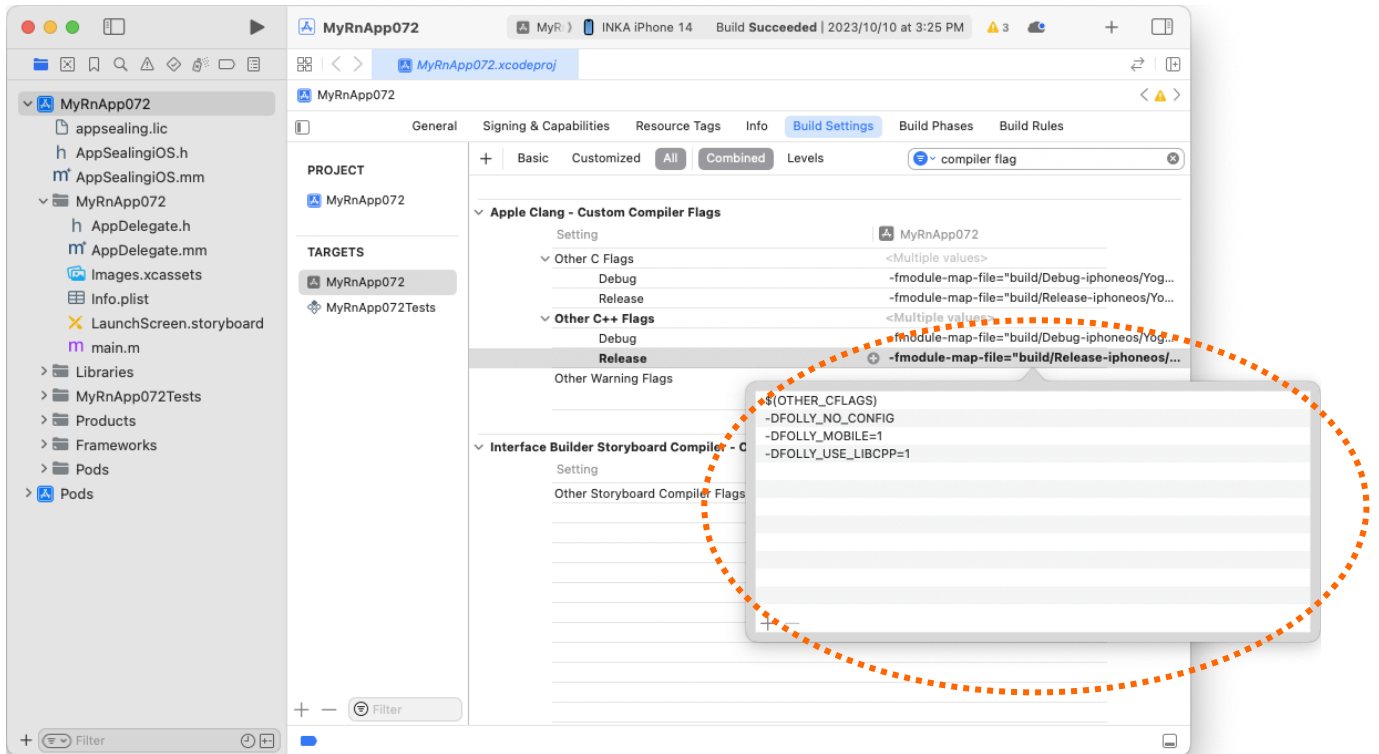
다시 Xcode 프로젝트로 이동해서 "TARGETS"의 프로젝트를 선택하고 "Build Settings" 탭을 선택합니다. 탭이 선택되면 왼쪽 아래의 "All" 옵션을 선택합니다.



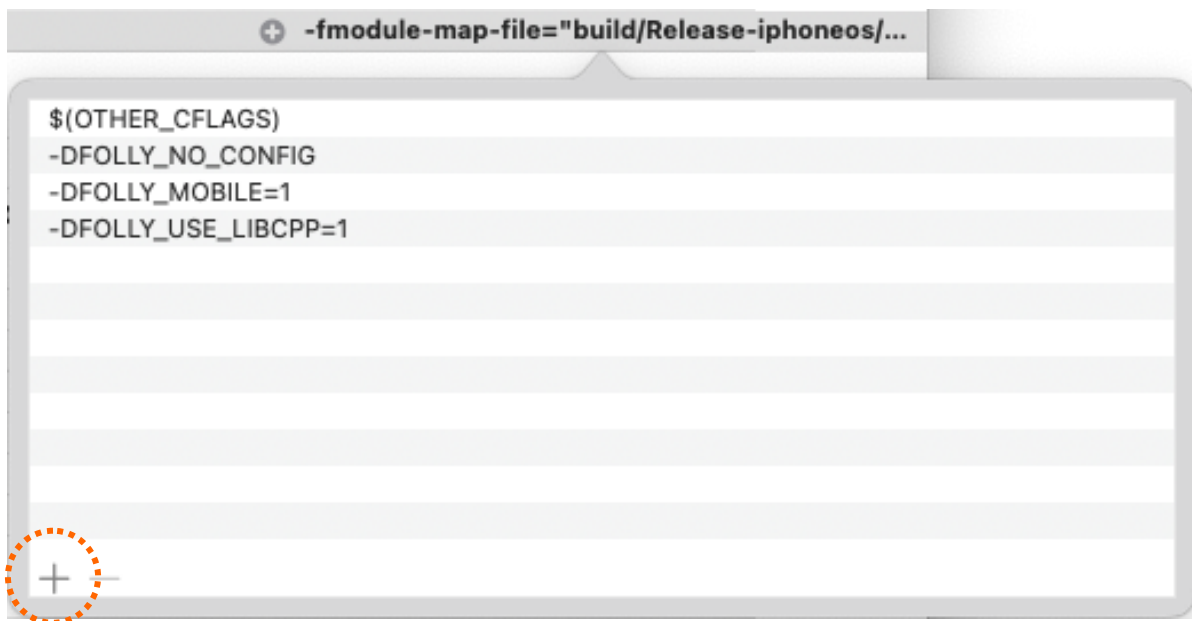
이제 오른 쪽에 있는 검색 상자에 "compiler flag"를 입력한 후 아래쪽에 검색 결과가 표시되면 그 중 "Other C++ Flags"와 그 밑에 "Release" 항목을 선택하고 설정값을 더블클릭합니다.



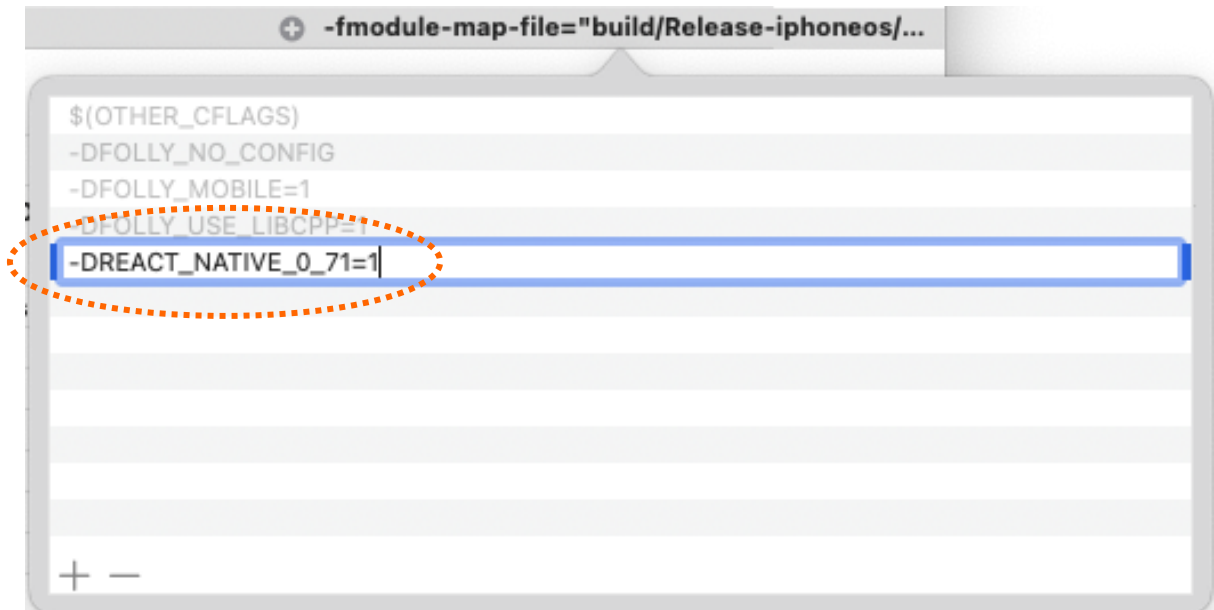
값을 더블클릭하면 아래와 같이 컴파일 플래그 설정값들이 표시됩니다.



이제 창 하단에서 “+” 버튼을 클릭하고 “-DREACT_NATIVE_0_71=1” 값을 입력합니다. (큰 따옴표는 제외하고 입력해야 합니다).

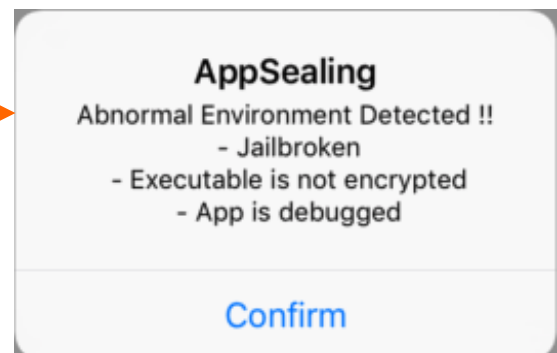


아래와 같이 창 내용이 변경되면 화면의 아무 곳이나 클릭해 창을 닫습니다.



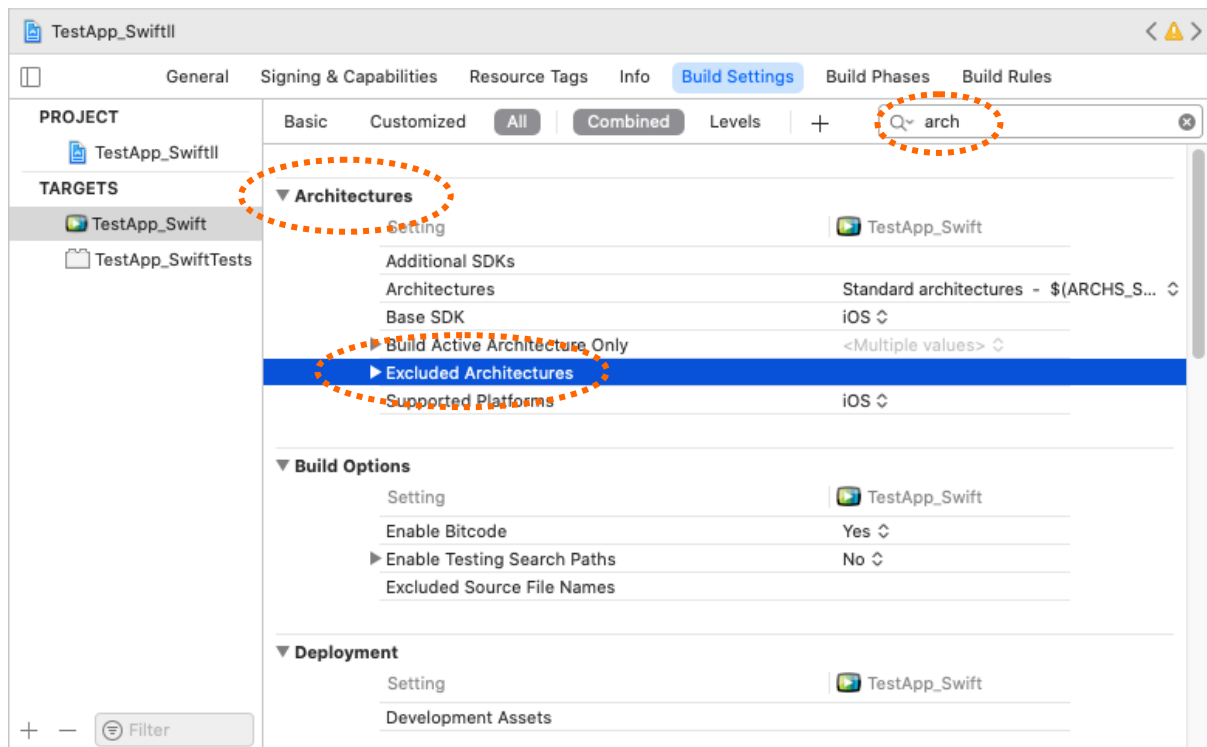
Xcode나 gdb를 사용하여 앱을 디버깅하거나 비정상적인 장치에서 앱을 실행하면 앱에서 아래와 같은 간단한 경고 상자가 표시됩니다. 이런 상황에서 사용자 조치와 상관없이 앱은 20초 후에 자동으로 종료됩니다.

iOS 기기에서 보안 침입이 감지되면 경고 창이 출력되고 20초 후 또는 사용자가 "Confirm" 버튼을 누르면 앱이 닫힙니다.



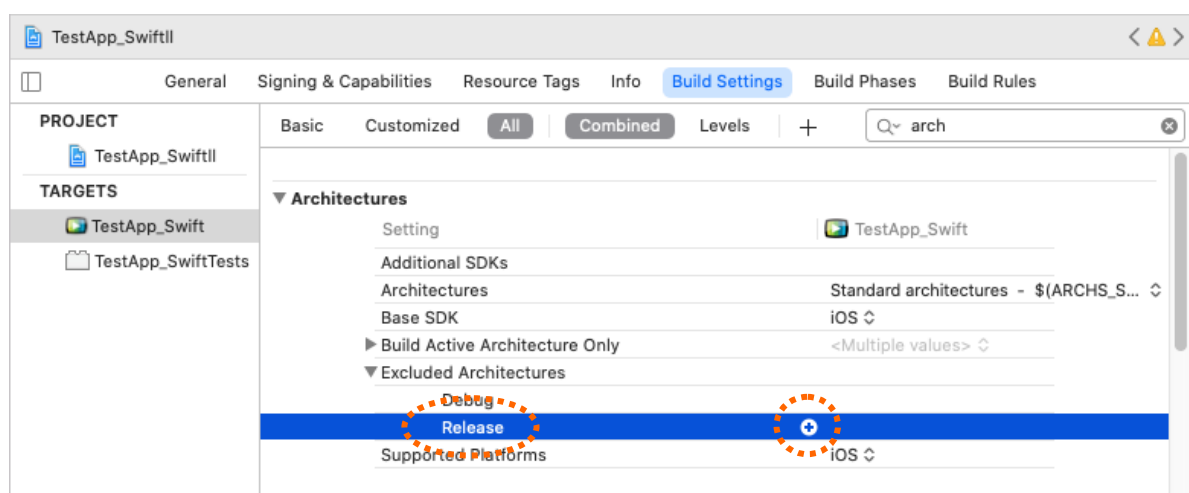
Part 4. "Build Settings" 수정

먼저 Build Settings에서 arch를 검색하십시오.

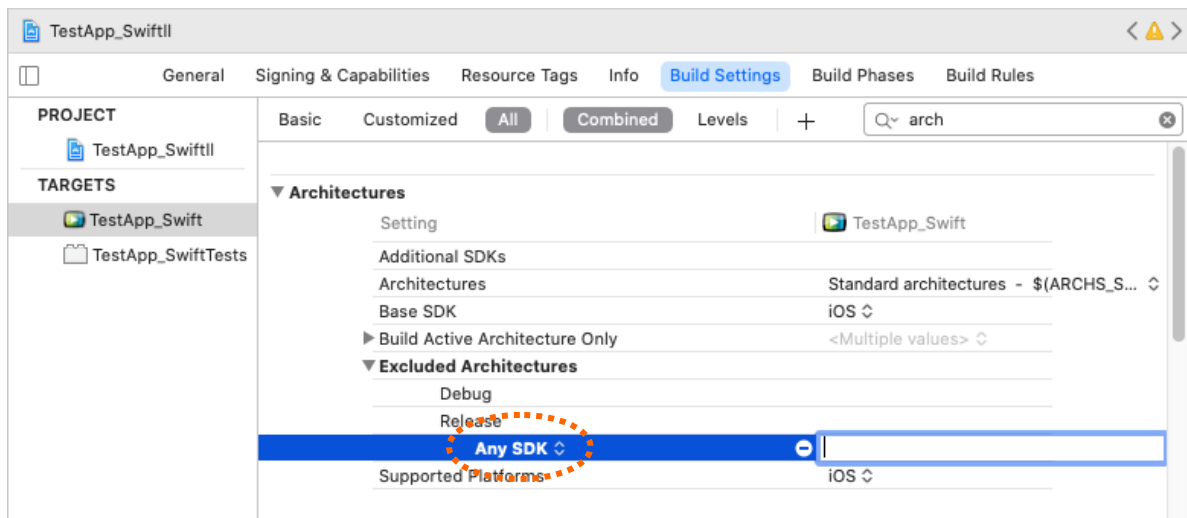


"Architectures" 필드가 나타나면 다음 단계를 수행하여 값을 설정하십시오.

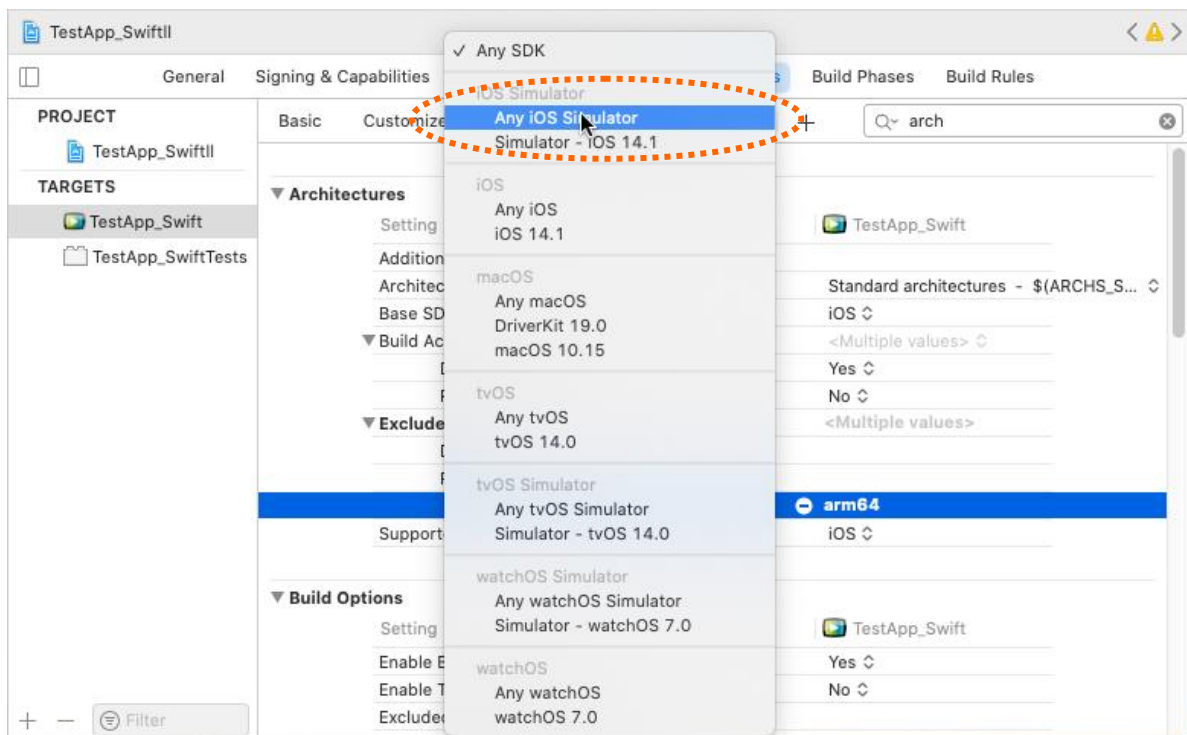
1) "Excluded Architectures" 왼쪽 삼각형 아이콘을 눌러 확장하십시오.



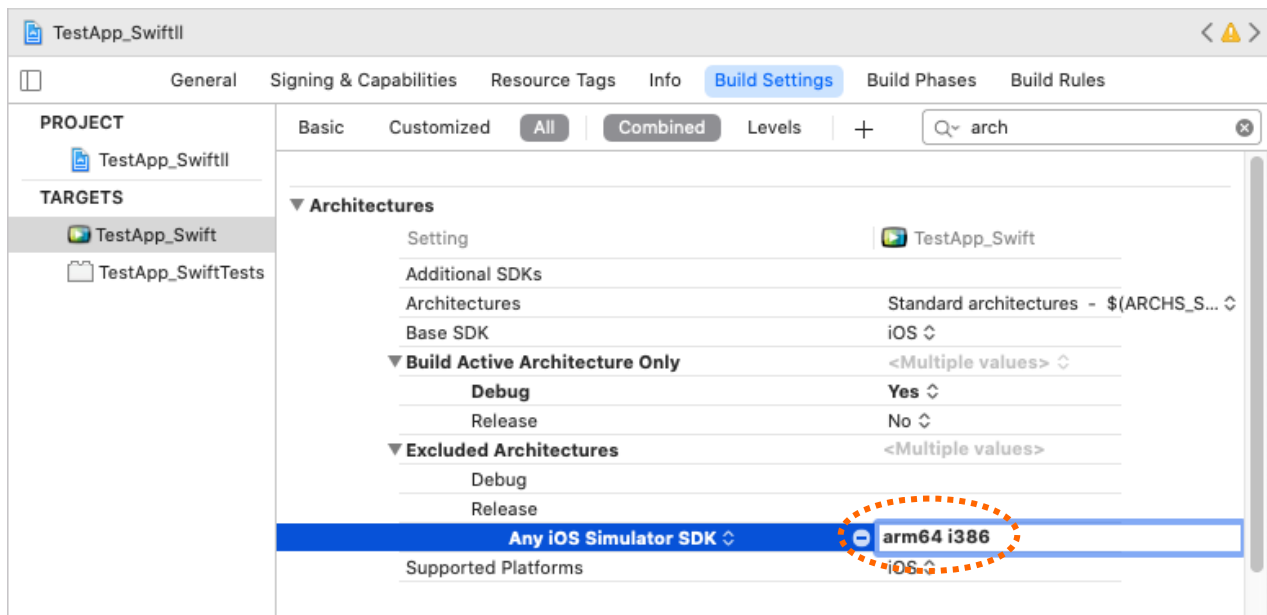
2) "Release" 항목을 선택하고 오른쪽의 "+" 아이콘을 클릭하십시오.



3) "Any SDK" 항목이 추가되면 클릭하여 팝업 메뉴를 띄우십시오.



4) 팝업 메뉴의 항목 중에서 **"Any iOS Simulator"**를 선택합니다.



5) "Any iOS Simulator SDK" 우측의 입력란에 **"arm64 i386"** 값을 입력합니다. (arm64 와 i386 사이는 공백이 들어갑니다)

이제 AppSealing 보안 기능이 프로젝트에 추가되었습니다. 평소와 같이 'Build', 'Run' 또는 'Archive'를 수행하십시오.

Part 5. 앱 빌드 및 후처리 과정

5-1 앱 빌드 모드에 따른 주의 사항

* React Native용 AppSealing SDK는 릴리스 모드에서만 동작합니다.

React Native 앱의 디버그 모드 불안정성으로 인해 AppSealing 보안 기능은 Release 모드 빌드에서만 적용되고 아래의 기능들이 활성화됩니다. 개발 단계에서 디버그 모드 빌드를 위해서는 AppSealing 적용을 비활성화 하고 진행하세요.

- 탈옥 감지 : Jailbreak detection
- 안티 디버깅 : Anti-debugging
- 안티 후킹 : Anti-hooking
- 실행 파일 암호화 감지 : Not encrypted executable file detection
- 앱 무결성 감지 : App-Integrity corruption detection
- 앱 재서명 감지 : Re-signing detection

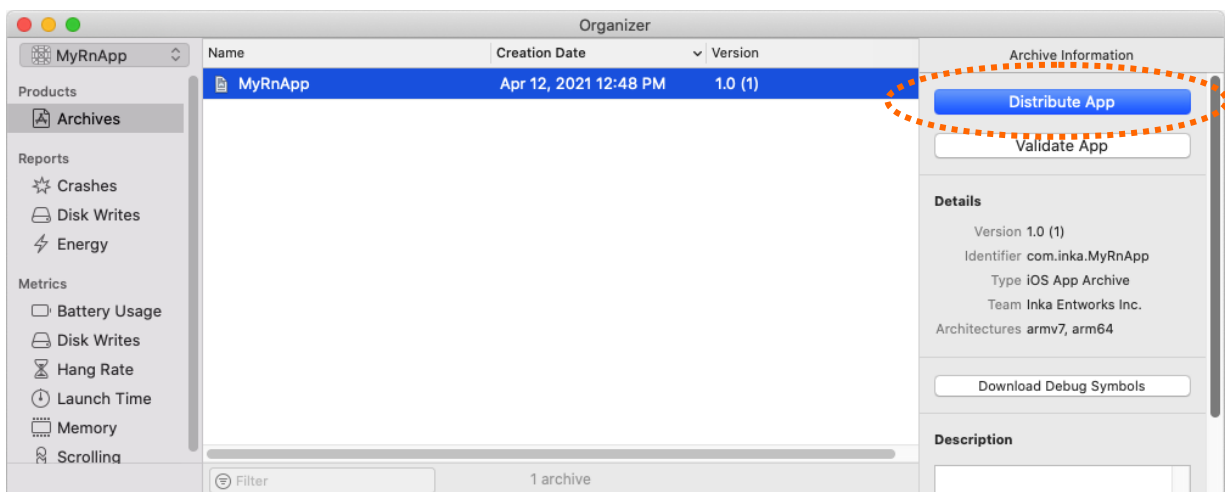
앱 스토어에 배포할 때 앱을 릴리스 모드로 빌드 하십시오. 릴리스 모드로 AppSealing 을 테스트 할 경우 앱스토어 또는 TestFlight로 앱을 배포해야 합니다. 그렇지 않으면 실행 파일이 암호화되지 않은 것이 탐지되거나 앱이 변조된 것으로 탐지되어 일정 시간 후 앱이 종료됩니다.

5-2 앱 무결성 및 인증서 검증 정보 생성

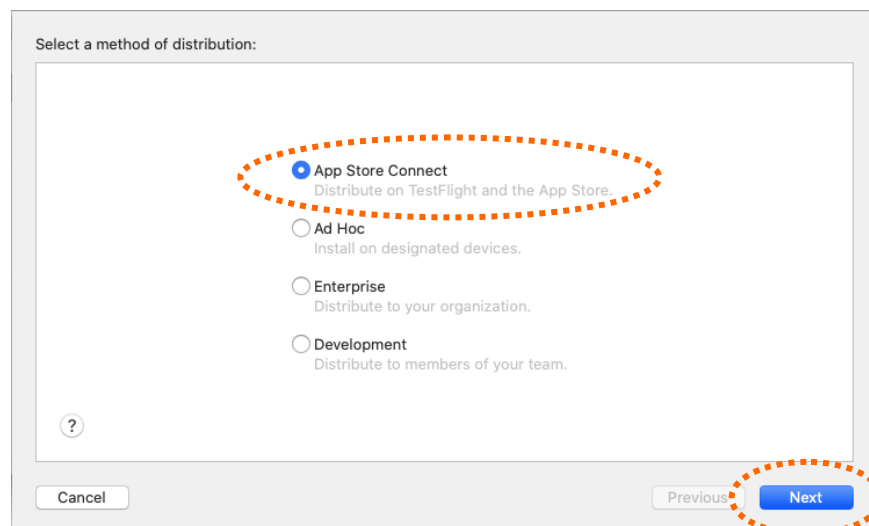
앱을 테스트 하거나 스토어에 배포할 경우 앱의 무결성 검증을 위한 추가 작업을 진행해야 합니다. 이 단계를 건너뛰는 경우 단말에서 앱을 실행하면 무결성 검증에 실패하여 일정 시간 후 앱이 종료됩니다.

TestFlight나 App Store 배포 시, 혹은 AdhocEnabled SDK를 사용하여 Ad Hoc 배포 시에는 반드시 이 단계를 수행해야 합니다.

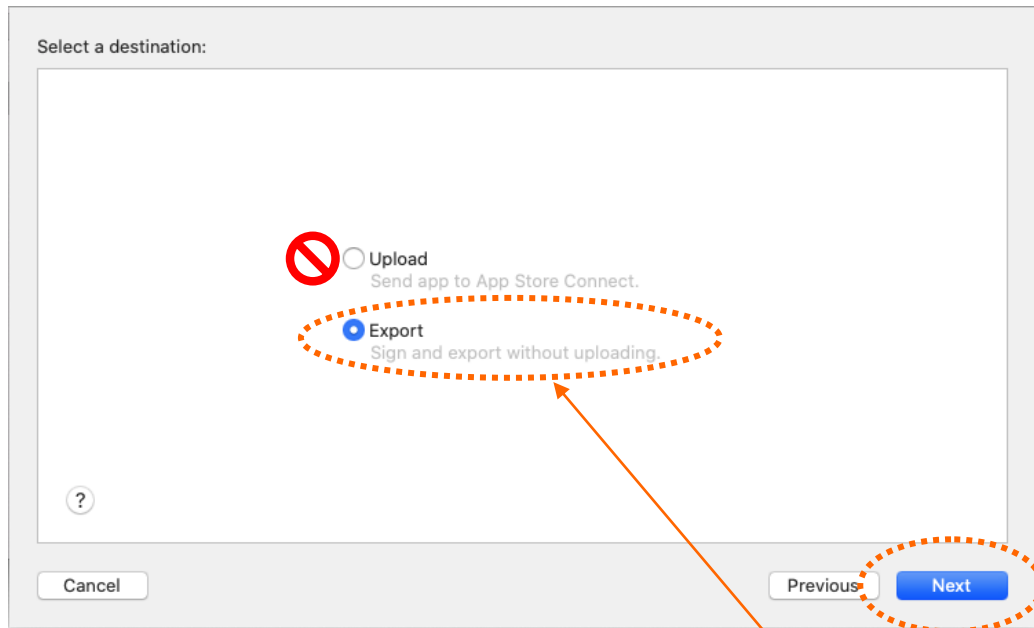
앱스토어나 TestFlight에 앱을 업로드 하기 위한 과정을 단계 별로 살펴 보겠습니다. 아래는 Archive가 완료된 상태의 Organizer 화면입니다.



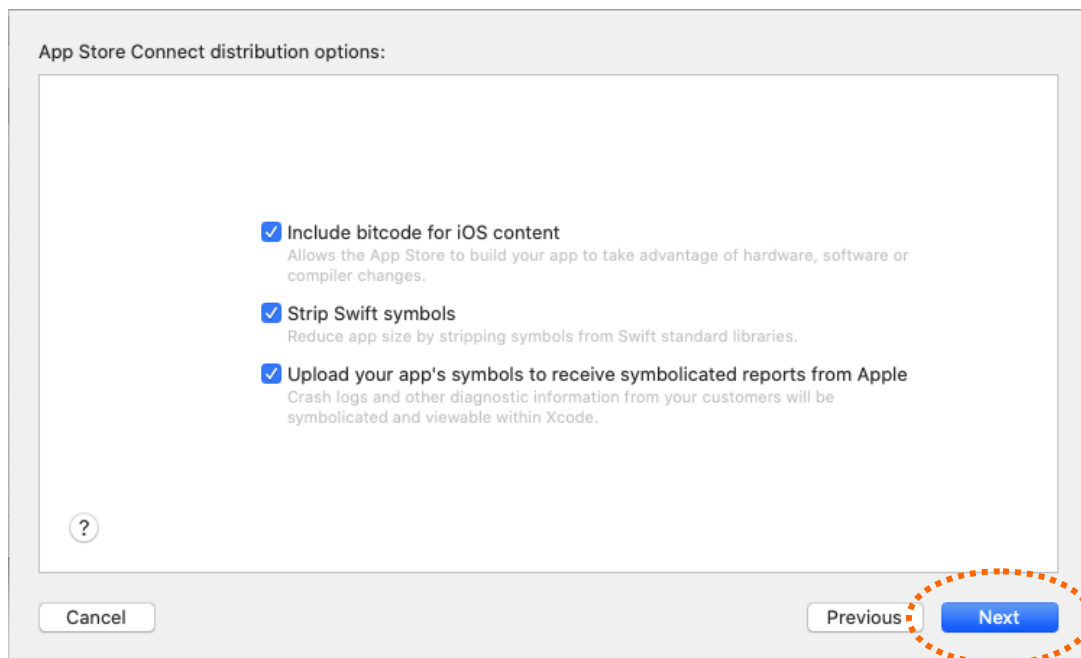
이 상태에서 업로드 할 IPA를 생성하기 위해 "Distribute App" 버튼을 클릭합니다.



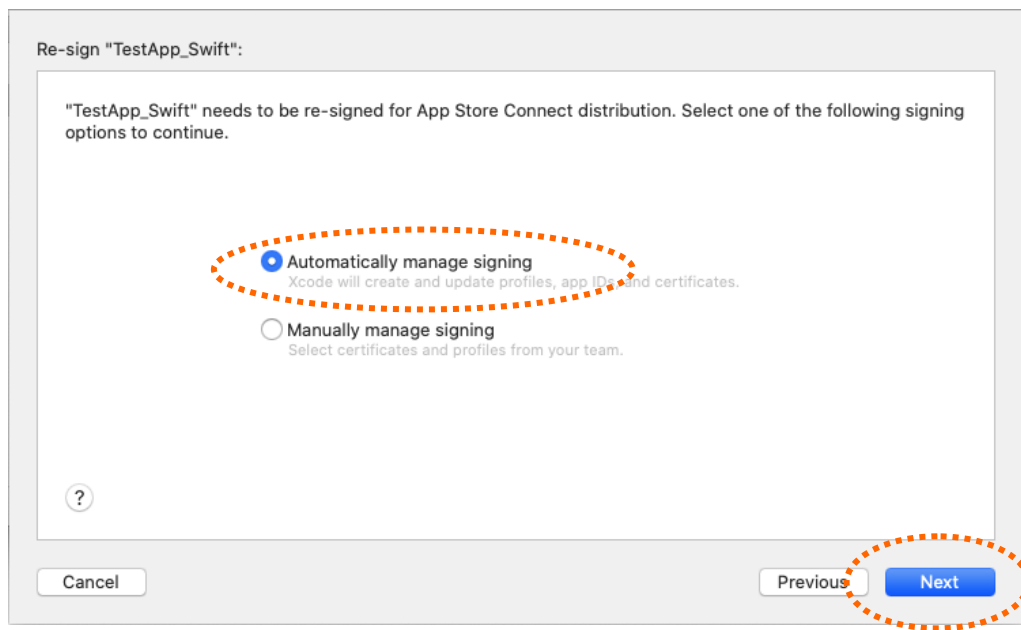
"App Store Connect"가 선택된 상태에서 "Next" 버튼을 클릭합니다. (AdhocEnabled SDK를 사용하는 경우 Ad Hoc도 선택이 가능합니다.)



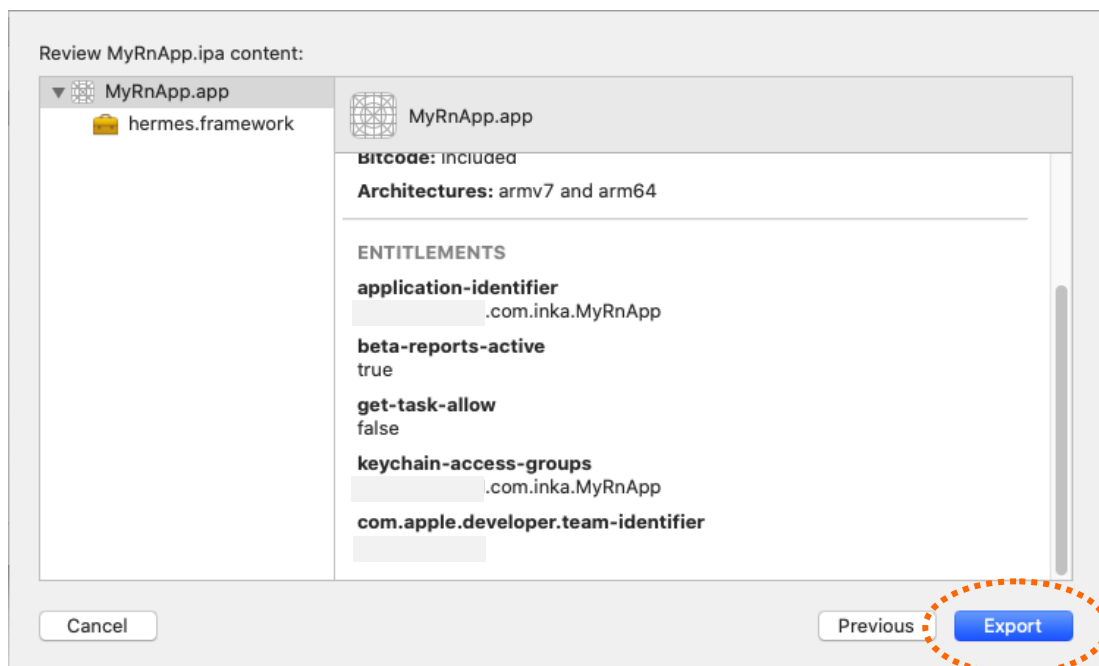
기존에는 "Upload"를 선택하는 경우가 대부분이지만 **AppSealing 적용을 위해서는 반드시 "Export"를 선택**해야 합니다. 이는 업로드 전 무결성 및 인증서 스냅샷 처리 작업이 필요하기 때문으로 **이 과정이 잘못되거나 생략되면 앱이 정상적으로 실행되지 않으므로 유의해야 합니다.** "Export"가 선택된 상태에서 "Next" 버튼을 클릭합니다.



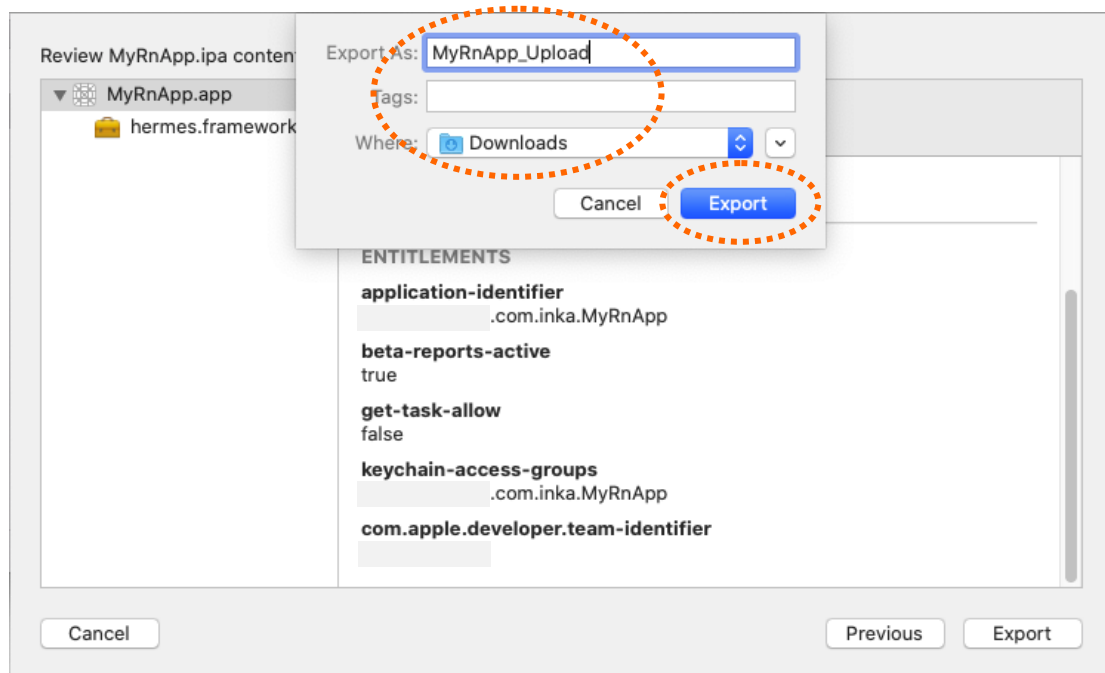
옵션 선택은 그대로 둔 상태로 "Next" 버튼을 클릭합니다.



옵션 선택은 그대로 둔 상태로 “Next” 버튼을 클릭합니다. 이제 아래와 같이 IPA 파일로 export 할 수 있는 창이 나타납니다.

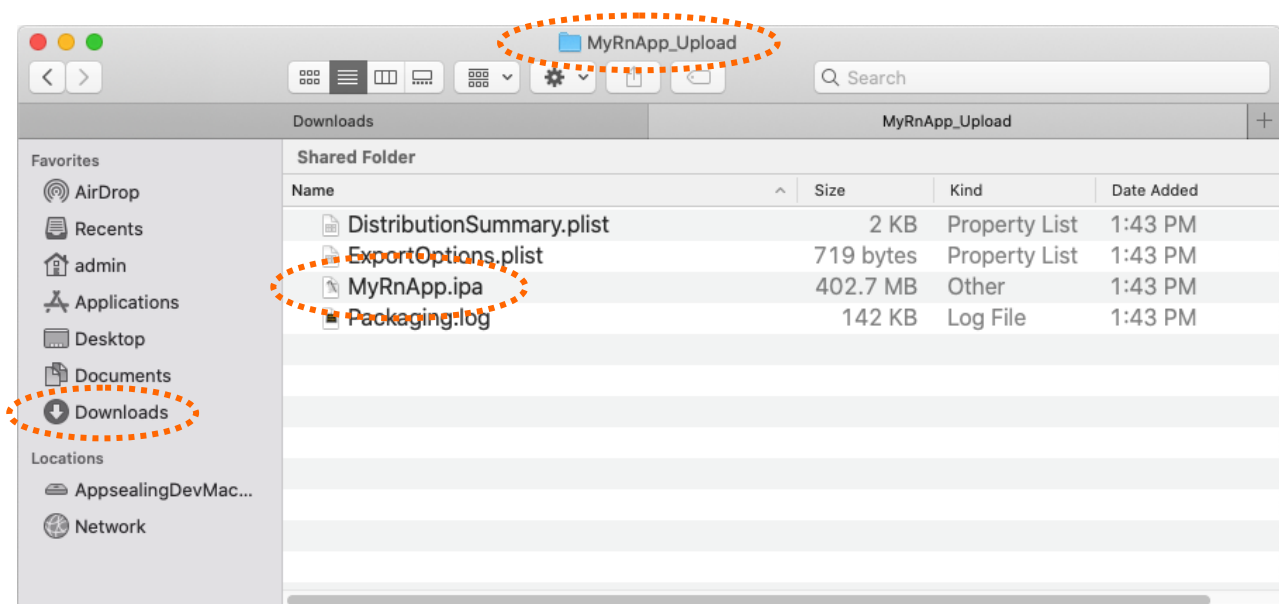


내용을 확인한 후 “Export” 버튼을 클릭합니다.



Export 위치를 지정하는 창이 표시되면 저장 위치를 선택하고 “Export”를 클릭합니다. 여기서는 ~/Downloads/MyRnApp_Upload 라는 이름의 폴더를 사용했습니다.

“Export”를 클릭하면 해당 폴더에 IPA 파일이 생성됩니다. 아래와 같이 Finder에서 확인할 수 있습니다. 이제 이 폴더의 위치를 잘 기억하거나 또는 Finder에서 해당 폴더로 이동한 후 창을 열어 놓습니다.

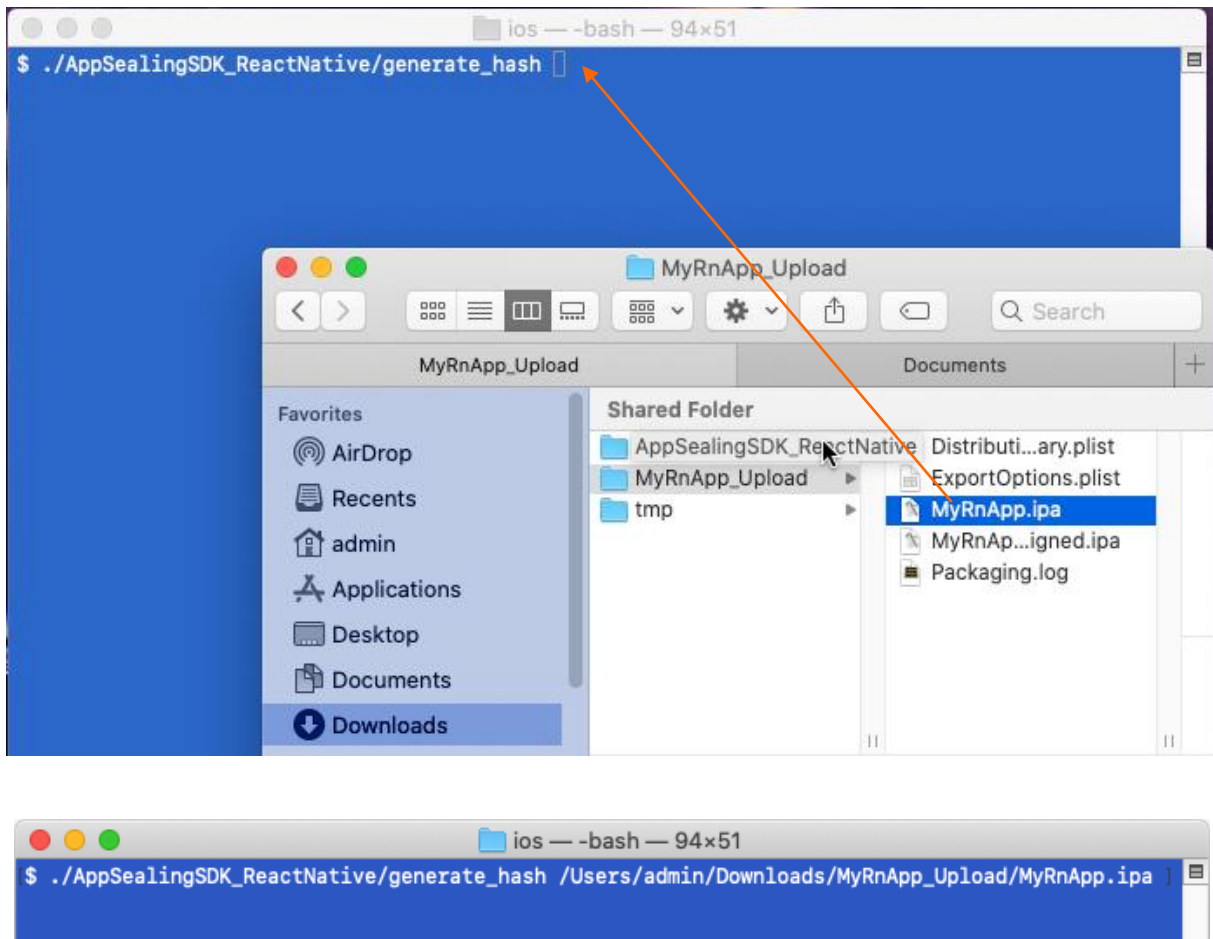


다음 명령으로 Tools 폴더 안의 파일들에 실행 권한을 부여합니다.

```
$ chmod +x MyRnApp/ios/AppSealingSDK_ReactNative/generate_hash
```

앞에서 실행 권한을 부여한 generate_hash 스크립트를 아래와 같이 실행합니다. 파라미터는 이전 단계에서 Export 했던 IPA 파일의 경로입니다. 경로를 직접 타이핑 하지 않고 열어 두었던 Finder 창에서 MyRnApp.ipa 파일을 터미널 창으로 드래그&드롭 해도 됩니다.

```
$ ./AppSealingSDK_ReactNative/generate_hash ~/Downloads/MyRnApp_Upload/MyRnApp.ipa
```



스크립트를 실행하면 아래 화면과 같이 진행 상황이 표시되면서 지정한 IPA에 무결성 검증 정보와 인증서 검증 정보를 추가하고 JavaScript 바이트 코드를 암호화 합니다.

```
ios — bash — 94x51
$ ./AppSealingSDK_ReactNative/generate_hash /Users/admin/Downloads/MyRnApp_Upload/MyRnApp.ipa

+-----+
| AppSealing IPA Hash Generator V1.0.3 : provided by INKA Entworks |
+-----+

[Target IPA]      = /Users/admin/Downloads/MyRnApp_Upload/MyRnApp.ipa
[URL Scheme]     =

[Working Directory] = /var/folders/vd/qcxck8j5171d3ztm4m0wzjb80000gn/T/AppSealing/ffd2a28add28c8c007af369a5e7024c5/

1. Payload has extracted from the IPA ...
2. Trying to receive encryption key from AppSealing server ...
3. Successfully received encryption key ...
4. Generating app integrity/certificate snapshot ...
Executable=/private/var/folders/vd/qcxck8j5171d3ztm4m0wzjb80000gn/T/AppSealing/ffd2a28add28c8c007af369a5e7024c5/Package/Payload/MyRnApp.app/MyRnApp
5. Encrypting app integrity/certificate snapshot ...
6. Inserting app integrity/certificate snapshot into IPA ...
7. Encrypting license file ...
8. Encrypting React Native javascript bytecode file ...
==> Processing for sealing ..... Done!
3. Successfully encrypted javascript bytecode ...
9. Codesigning your app using certificate used to sign your IPA ...
Executable=/private/var/folders/vd/qcxck8j5171d3ztm4m0wzjb80000gn/T/AppSealing/ffd2a28add28c8c007af369a5e7024c5/Package/Payload/MyRnApp.app/MyRnApp
/var/folders/vd/qcxck8j5171d3ztm4m0wzjb80000gn/T/AppSealing/ffd2a28add28c8c007af369a5e7024c5/Package/Payload/MyRnApp.app/: replacing existing signature
10. Rebuilding & re-signing IPA ...

>>> All processes have done successfully .....

$
```

이 과정은 "Ad Hoc", "Enterprise", "Development"로 배포할 경우에도 동일하게 적용됩니다.

5-3 메소드 스위즐링 / 메소드 후킹 감지 기능 제어

메소드 스위즐링은 특정 클래스의 기존 메소드의 기능을 새로운 메소드의 기능으로 교환하여 원래의 메소드를 호출할 때 실제로는 새로운 메소드의 코드가 실행되도록 하는 기술로 주로 Objective-C에서 사용되며 이를 통해 기존 코드의 동작을 유연하게 수정하거나 확장할 수 있게 해줍니다.

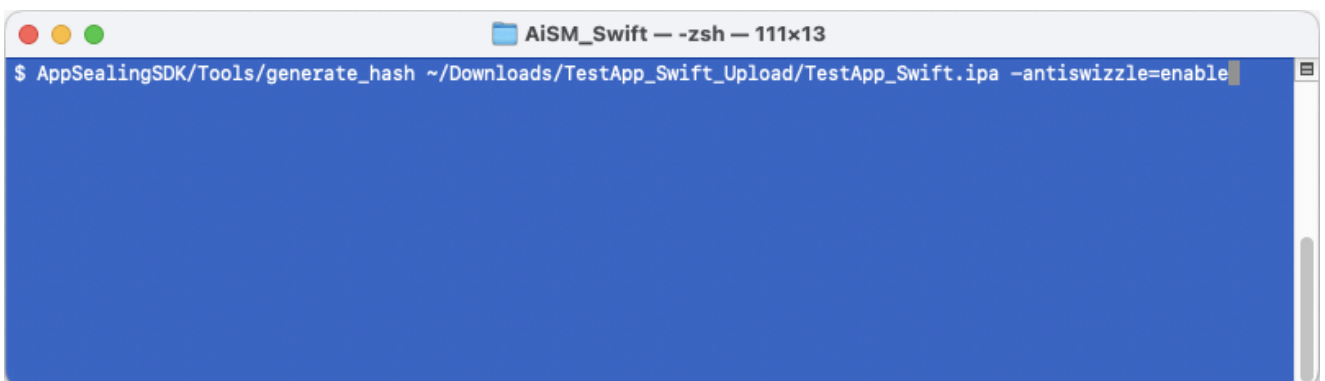
메소드 스위즐링은 유용한 기술이지만 공격자는 메소드 스위즐링을 통해 기존의 메소드를 자신의 악성 코드로 교환할 수도 있습니다. 예를 들어 사용자의 입력을 처리하는 메소드를 수정하여 데이터를 탈취하거나, 원하지 않는 행동을 실행하도록 만들 수 있습니다.

또한 시스템의 로깅 메소드를 스위즐링하여, 중요한 이벤트가 기록되지 않도록 할 수 있고 이를 통해 악의적인 행동이 감지되지 않게 할 수 있습니다.

AppSealing SDK에는 이러한 메소드 스위즐링 동작을 감지할 수 있는 기능이 추가되어 있지만 메소드 스위즐링이 항상 공격의 수단으로만 사용되는 것이 아니기에 이를 감지할 것인지 하지 않을 것인지를 직접 지정할 수 있습니다.

기본적으로 오동작을 막기 위해 메소드 스위즐링 감지 기능이 비활성화 된 상태로 SDK가 배포되지만 generate_hash 스크립트를 실행하는 과정에서 이를 활성화 시킬 수 있습니다. 만약 앱 테스트 단계에서 메소드 스위즐링이 감지되었다는 보안 경고가 발생할 경우 앱에 포함된 다른 라이브러리에서 의도적으로 메소드 스위즐링을 사용하는 경우가 있을 수 있기 때문에 이 경우는 메소드 스위즐링 감지를 활성화 시키면 안 됩니다.

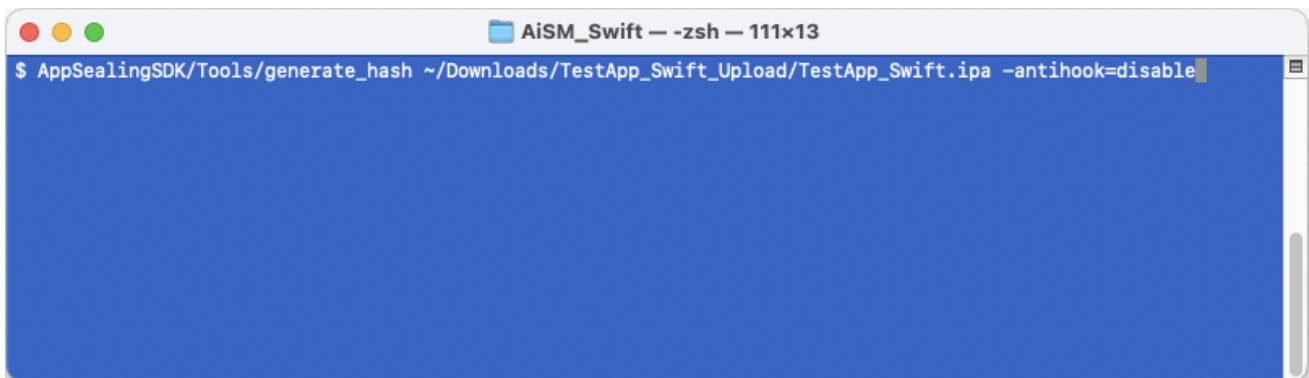
generate_hash 스크립트 실행 시 메소드 스위즐링 감지 기능을 활성화 하려면 다음과 같이 **antiswizzle=enable** 파라미터를 추가한 상태로 generate_hash를 실행하면 됩니다.

A screenshot of a macOS terminal window titled "AiSM_Swift - zsh - 111x13". The terminal shows a command being executed: "\$ AppSealingSDK/Tools/generate_hash ~/Downloads/TestApp_Swift_Upload/TestApp_Swift.ipa -antiswizzle=enable". The terminal background is blue, and the text is white. The command is entered at the prompt, and the rest of the terminal is empty.

```
$ AppSealingSDK/Tools/generate_hash ~/Downloads/TestApp_Swift_Upload/TestApp_Swift.ipa -antiswizzle=enable
```

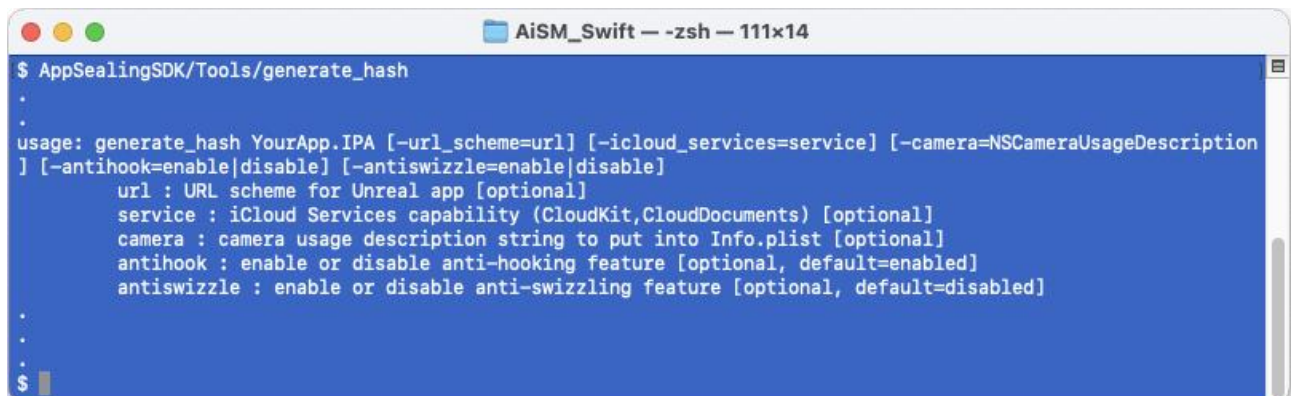
메소드 스위즐링과 함께 사용되는 공격 방법으로 메소드 후킹이 있습니다. 메소드 후킹은 메소드 스위즐링과 유사하지만 메소드를 서로 바꾸는 것이 아니라 메소드의 실행 흐름을 직접 조작하고 제어한다는 차이점이 있습니다. 메소드 스위즐링은 필요에 따라 앱이나 특정 라이브러리에서 사용될 수 있지만 정상적인 앱과 라이브러리에서 메소드 후킹이 사용되는 경우는 없기 때문에 AppSealing SDK에는 메소드 후킹 감지 기능이 기본으로 활성화 되어 있지만 필요한 경우 이 기능을 비활성화 시킬 수 있습니다.

메소드 후킹 감지 기능을 비활성화 하려면 다음과 같이 `antihook=disable` 파라미터를 추가하여 `generate_hash` 스크립트를 실행하면 됩니다.



```
AiSM_Swift - zsh - 111x13
$ AppSealingSDK/Tools/generate_hash ~/Downloads/TestApp_Swift_Upload/TestApp_Swift.ipa -antihook=disable
```

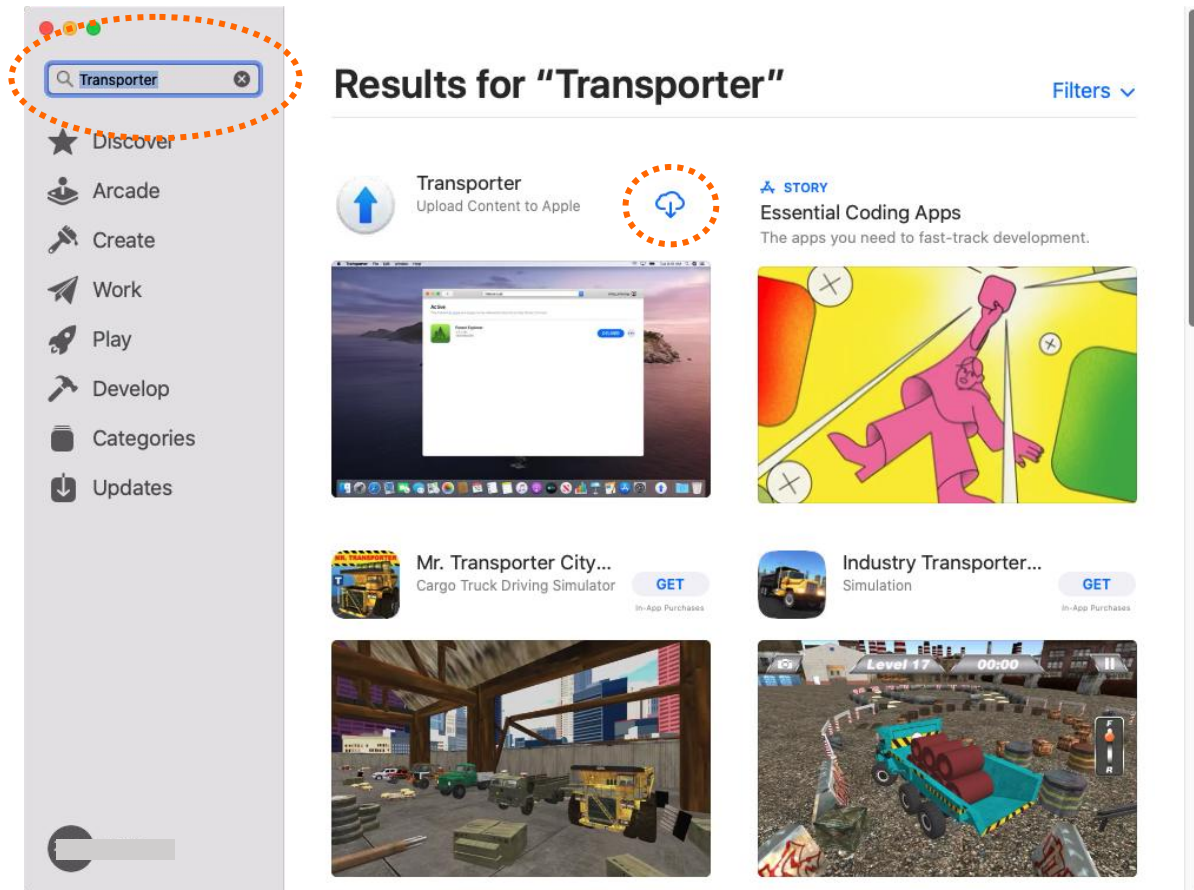
`generate_hash` 스크립트의 모든 파라미터를 확인하고 싶을 경우는 IPA 경로명 없이 스크립트만 실행하면 아래 화면과 같이 사용 가능한 모든 파라미터를 확인할 수 있습니다.



```
AiSM_Swift - zsh - 111x14
$ AppSealingSDK/Tools/generate_hash
.
.
usage: generate_hash YourApp.IPA [-url_scheme=url] [-icloud_services=service] [-camera=NSCameraUsageDescription]
[-antihook=enable|disable] [-antiswizzle=enable|disable]
url : URL scheme for Unreal app [optional]
service : iCloud Services capability (CloudKit,CloudDocuments) [optional]
camera : camera usage description string to put into Info.plist [optional]
antihook : enable or disable anti-hooking feature [optional, default=enabled]
antiswizzle : enable or disable anti-swizzling feature [optional, default=disabled]
.
.
$
```

5-4 재서명 앱 App Store Connect 업로드

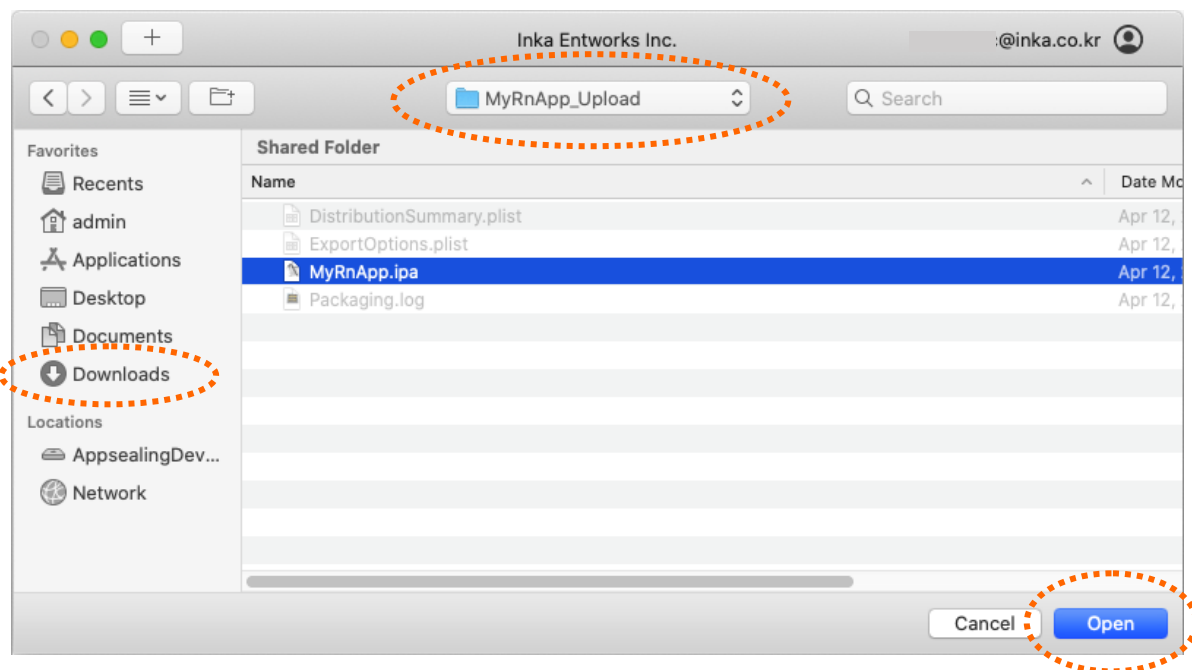
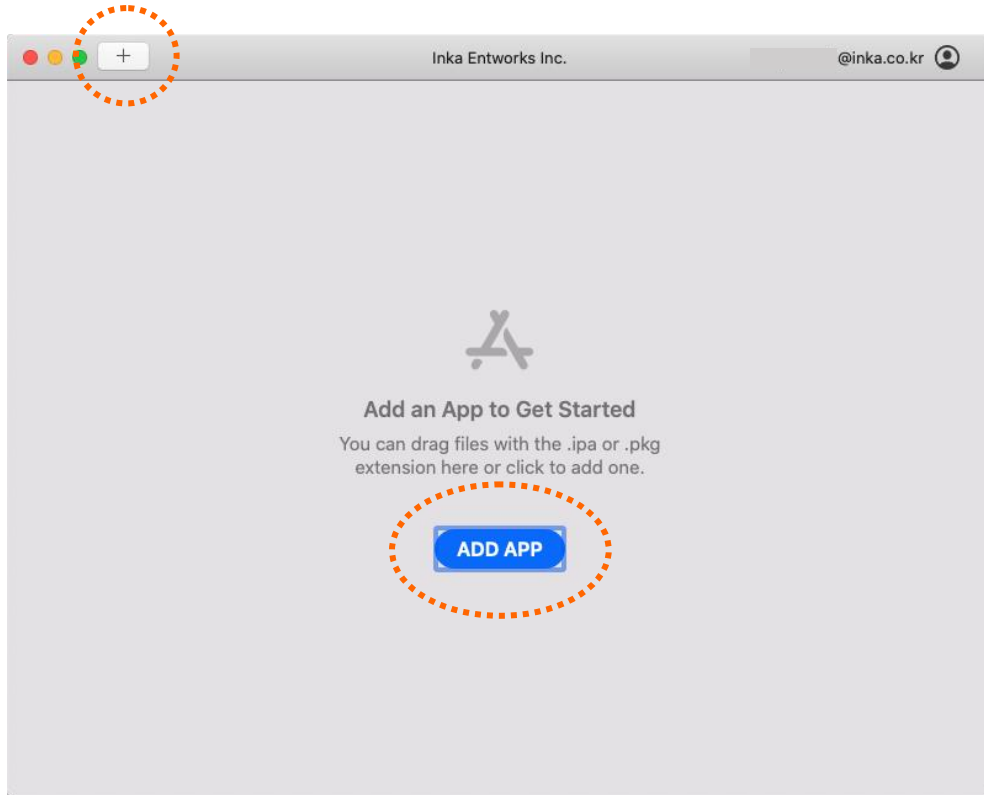
이제 재서명이 완료된 IPA를 App Store Connect에 업로드 합니다. (물론 AdhocEnabled SDK를 사용한 Ad Hoc 배포라면 이 과정이 필요 없습니다.) 간편한 업로드를 위해 Transporter 앱을 사용하겠습니다. 맥 OS에 Transporter가 설치되어 있지 않은 경우 맥 AppStore를 실행하고 "Transporter"를 검색하여 설치합니다.



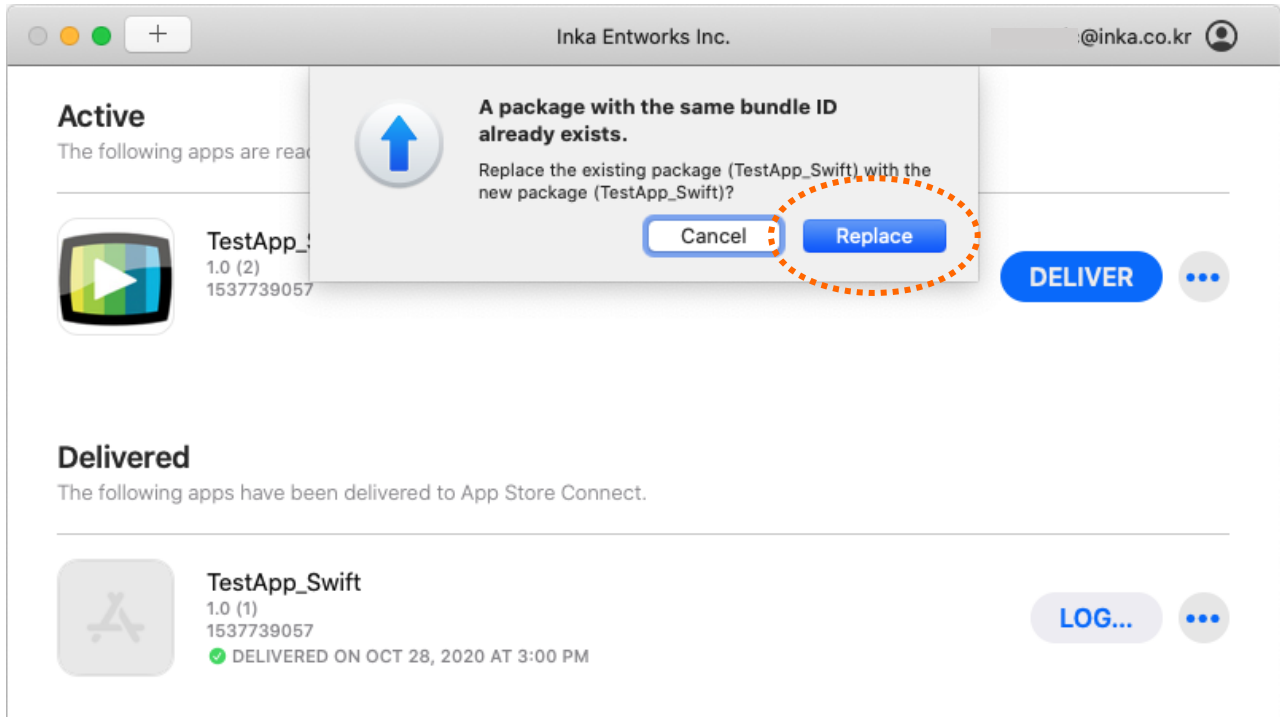
설치를 완료하고 실행하면 아래와 같이 Apple ID를 입력하는 창이 나타납니다. Apple 계정 ID와 비밀번호를 입력합니다. (이 과정은 최초 1회만 수행하면 됩니다)



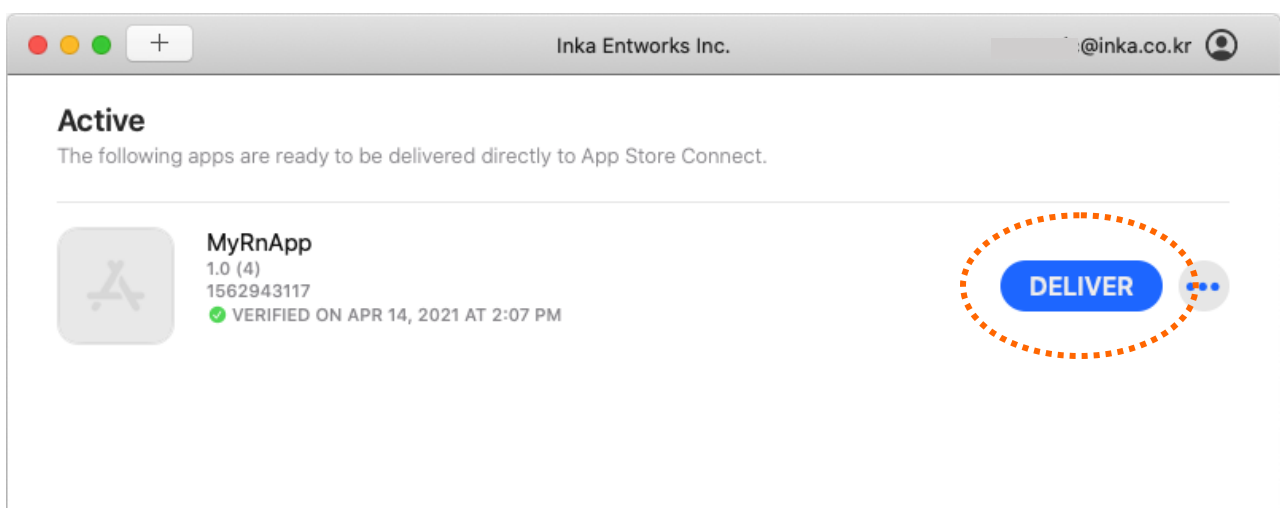
ID와 비밀번호를 입력하고 로그인 하면 아래와 같은 창이 나타나며 여기서 업로드 할 IPA를 선택하기 위해 좌측 상단의 + 버튼 또는 창 중앙의 "ADD APP" 버튼을 클릭하고 이전 단계에서 재서명이 완료된 IPA 파일을 선택합니다.

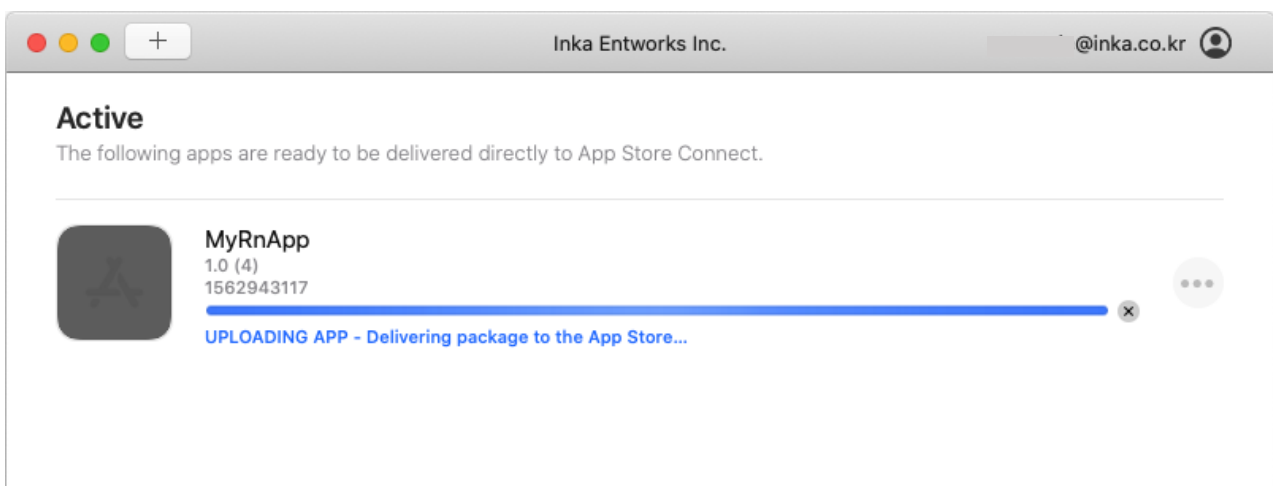
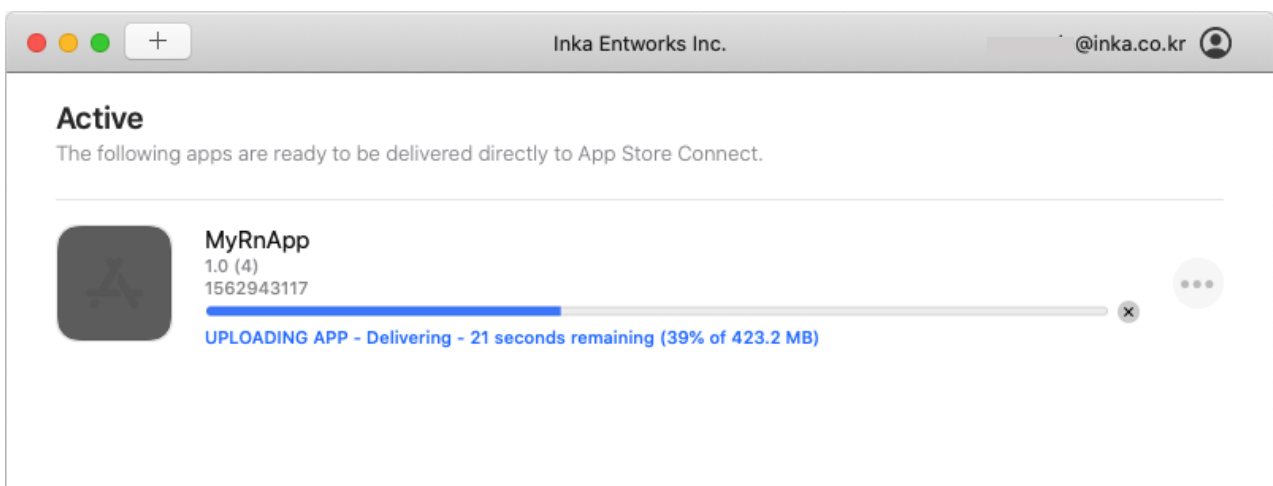
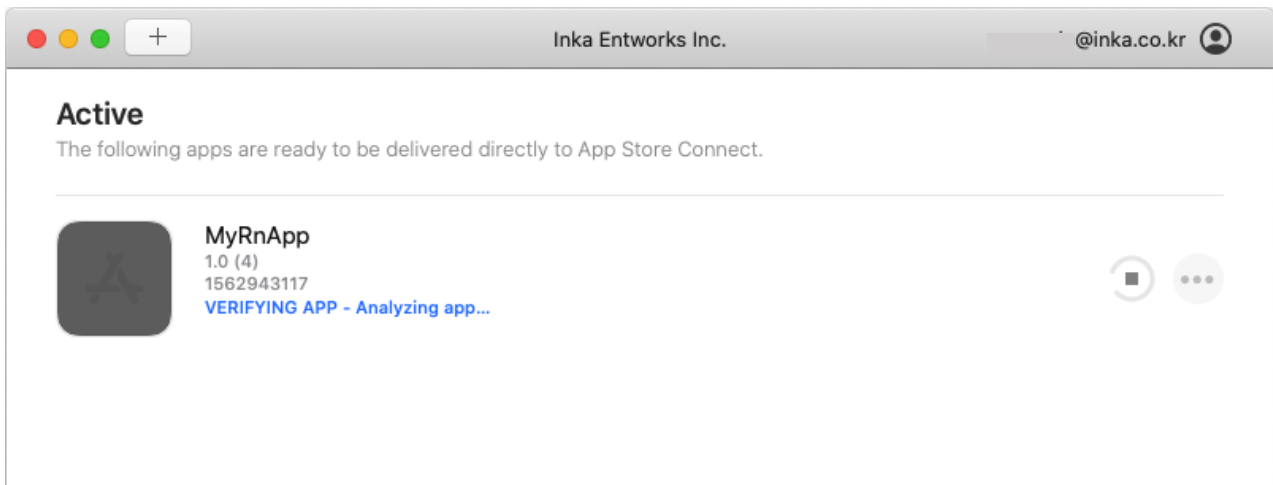


앱 최초 전송 이후 앱의 버전이나 빌드 번호를 증가시키고 새로 업로드 하려고 할 경우 IPA 파일을 추가하면 번들 ID가 동일하기 때문에 아래와 같은 확인 창이 나타날 수 있습니다. 이 경우 "Replace" 버튼을 클릭합니다.

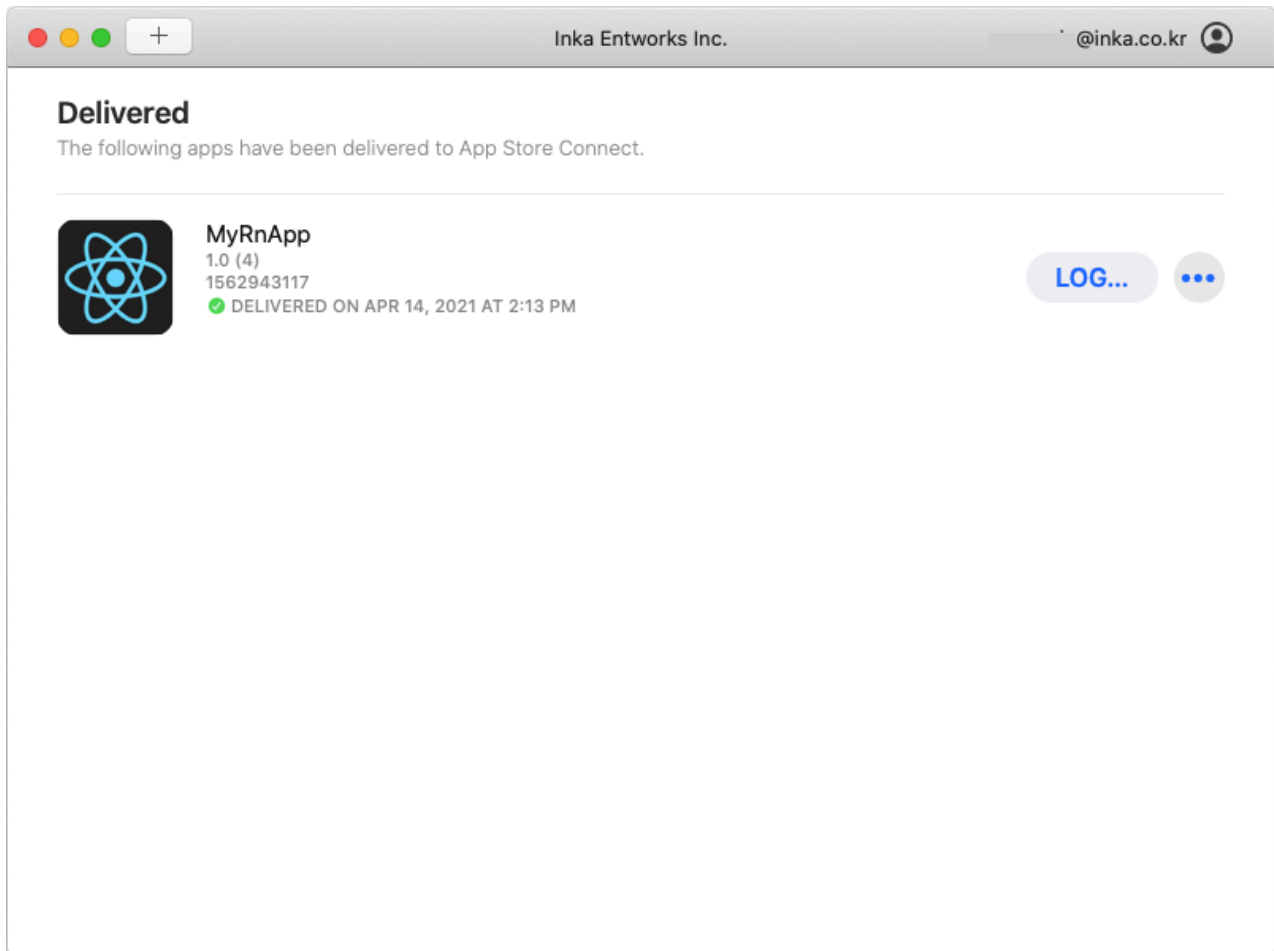


IPA가 추가된 상태에서 "DELIVER" 버튼을 클릭하면 앱 검증과 App Store Connect 업로드 과정이 진행됩니다.





아래 화면과 같이 전송이 완료되면 App Store Connect에서 심사를 위한 빌드 제출이나 TestFlight용 빌드 배포를 진행할 수 있습니다.



Part 6. AppSealing 기기 고유 ID 획득

AppSealing SDK는 기기 별로 고유 ID를 생성하고 관리합니다. AppSealing SDK를 사용하는 고객은 필요한 경우 AppSealing에서 제공하는 인터페이스를 이용하여 기기 고유 ID를 확인할 수 있습니다. 확인된 고유 ID는 AppSealing에서 제공하는 해킹 데이터 서비스와 연계하여 비즈니스에 활용할 수 있습니다.

6-1 기기 고유 ID 확인

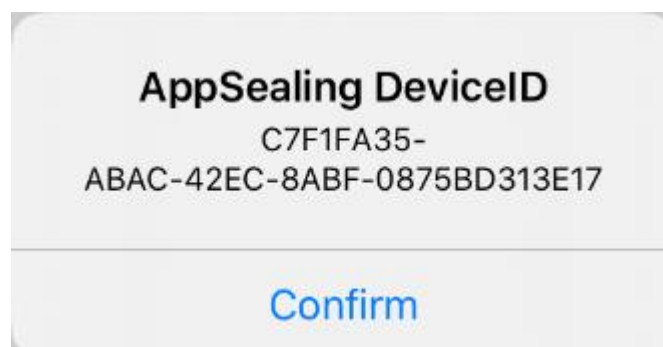
앞에서 오픈 한 AppDelegate.mm 파일의 didFinishLaunchingWithOptions 메소드의 "return YES;" 줄 바로 앞에 다음의 코드를 추가하고 파일 맨 윗 부분에 #include "AppsealingiOS.h"를 추가하면 AppSealing에서 사용하는 단말 고유 ID를 확인할 수 있습니다.

Simple UI code into 'AppDelegate.mm' for **React Native** project

```
NSString* msg = @"\n-----\n* AppSealing Device ID : ";
char _appSealingDeviceID[64];

if ( ObjC_GetAppSealingDeviceID( _appSealingDeviceID ) == 0 )
{
    UIAlertController *alert = [UIAlertController alertControllerWithTitle:@"AppSealing DeviceID"
        message:[NSString alloc] initWithUTF8String:_appSealingDeviceID
        preferredStyle:UIAlertControllerStyleAlert];
    UIAlertAction *confirm = [UIAlertAction actionWithTitle:@"Confirm"
        style:UIAlertActionStyleDefault
        handler:^(UIAlertAction * _Nonnull action) { }];
    [alert addAction:confirm];
    [self presentViewController:alert animated:YES completion:nil];
}
```

앱을 실행하면 아래와 같이 메시지 박스에서 기기 고유 ID를 확인할 수 있습니다.



Part 7. 문제 해결

7-1 Xcode Build error (1) Undefined symbols for architecture arm64: "_OBJC_CLASS_\$_AppSealingInterface"

```

Ld /Users/admin/Desktop/Build/Products/Debug-iphonios/TestApp_Swift.app/TestApp_Swift normal arm64
(in target: TestApp_Swift)
  cd /Users/admin/Documents/TestApp_Swift
  export IPHONEOS_DEPLOYMENT_TARGET=12.1
  /Applications/Xcode.app/Contents/Developer/Toolchains/XcodeDefault.xctoolchain/usr/bin/clang -
  arch arm64 -isysroot /Applications/Xcode.app/Contents/Developer/Platforms/iPhoneOS.platform/
  Developer/SDKs/iPhoneOS12.1.sdk -L/Users/admin/Desktop/Build/Products/Debug-iphonios -F/Users/
  admin/Desktop/Build/Products/Debug-iphonios -filelist /Users/admin/Desktop/Build/
  Intermediates.noindex/TestApp_Swift.build/Debug-iphonios/TestApp_Swift.build/Objects-normal/arm64/
  TestApp_Swift.LinkFileList -Xlinker -rpath -Xlinker @executable_path/Frameworks -miphoneos-
  version-min=12.1 -dead_strip -Xlinker -object_path_lto -Xlinker /Users/admin/Desktop/Build/
  Intermediates.noindex/TestApp_Swift.build/Debug-iphonios/TestApp_Swift.build/Objects-normal/arm64/
  TestApp_Swift_lto.o -Xlinker -export_dynamic -Xlinker -no_deduplicate -fembed-bitcode-marker -
  fobjc-link-runtime -L/Applications/Xcode.app/Contents/Developer/Toolchains/
  XcodeDefault.xctoolchain/usr/lib/swift/iphonios -Xlinker -add_ast_path -Xlinker /Users/admin/
  Desktop/Build/Intermediates.noindex/TestApp_Swift.build/Debug-iphonios/TestApp_Swift.build/
  Objects-normal/arm64/TestApp_Swift.swiftmodule -lStaticAppSec_Debug -L/Users/admin/Documents/
  TestApp_Swift/AppSealingSDK/Libraries -Xlinker -dependency_info -Xlinker /Users/admin/Desktop/
  Build/Intermediates.noindex/TestApp_Swift.build/Debug-iphonios/TestApp_Swift.build/Objects-normal/
  arm64/TestApp_Swift_dependency_info.dat -o /Users/admin/Desktop/Build/Products/Debug-iphonios/
  TestApp_Swift.app/TestApp_Swift

Undefined symbols for architecture arm64:
  "_OBJC_CLASS_$_AppSealingInterface", referenced from:
    objc-class-ref in ViewController.o
ld: symbol(s) not found for architecture arm64
clang: error: linker command failed with exit code 1 (use -v to see invocation)

symbol(s) not found for architecture arm64
! linker command failed with exit code 1 (use -v to see invocation)
✓ Copy TestApp_Swift.swiftmodule 0.1 seconds
✓ Copy TestApp_Swift.swiftdoc 0.1 seconds
! Build failed 31/01/2019, 3:38 PM 0.9 seconds
1 error
  
```

이 링크 오류는 두가지 경우에 발생할 수 있습니다. 첫번째는 macOS 11.x 환경에서 프로젝트를 생성하고 macOS를 12.x로 업데이트 한 후 다시 빌드하려고 할 경우이고 두번째는 프로젝트에 필요한 파일이 포함되지 않았을 경우입니다.

1) macOS 업데이트로 인해 링크 오류가 발생하는 경우의 해결방법 (0.66만 해당)

macOS 11.x 환경에서 React Native 0.66 버전의 프로젝트를 생성하고 macOS를 12.x, Xcode를 13.3 이상으로 업데이트 한 경우 기존의 프로젝트를 빌드하려고 하면 위와 같은 링크 오류가 발생합니다.

이 경우 새로운 macOS 환경에서 React Native 0.66 프로젝트를 새로 생성하고 기존 프로젝트의 코드를 그대로 옮겨와서 빌드해야 문제가 해결됩니다.

macOS 11.x 버전을 그대로 사용하거나 React Native 0.64, 0.65 버전 프로젝트인 경우는 해당 사항이 없습니다.

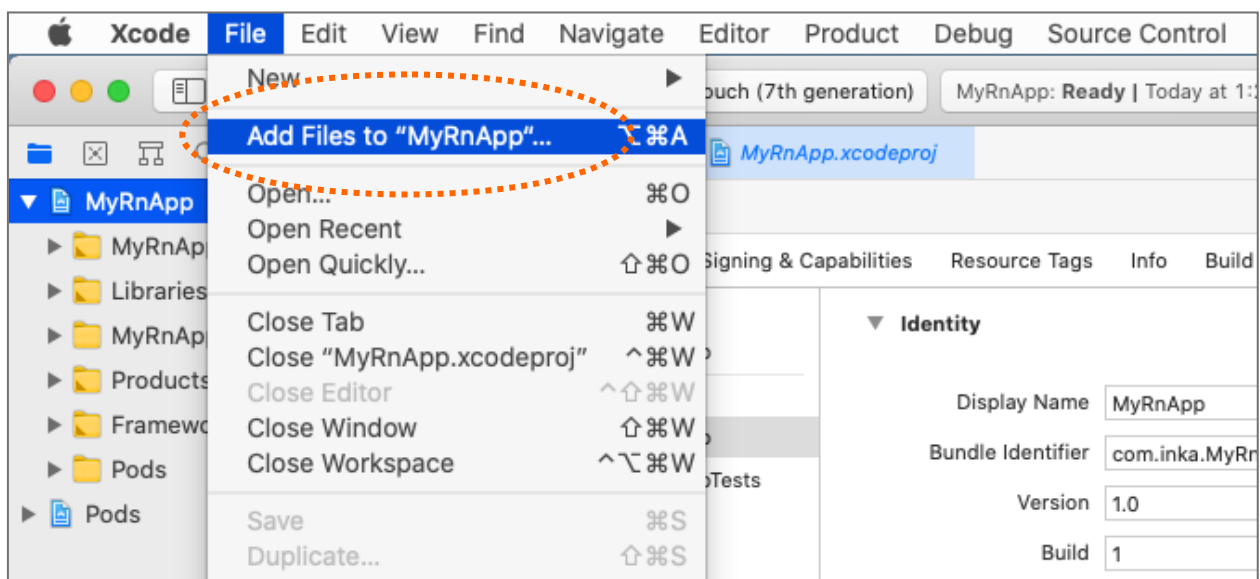
향후 프로젝트 코드 이전 없이 문제를 해결할 수 있도록 개선될 예정입니다.

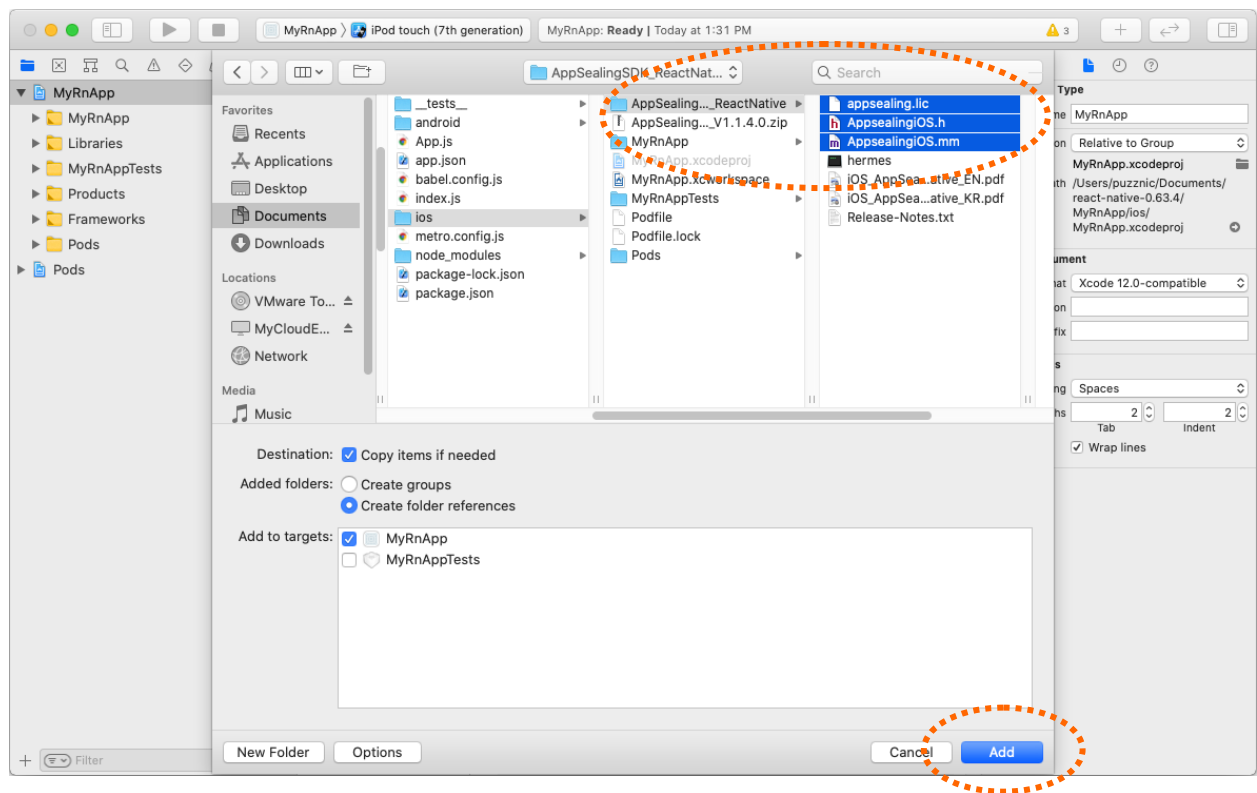
2) 파일이 포함되지 않아 발생하는 링크 오류의 해결방법

이 경우는 "AppSealingiOS.mm"이 프로젝트에 포함되어 있지 않을 때 발생합니다. 프로젝트 트리에서 해당 파일이 추가되어 있는지 확인하십시오.



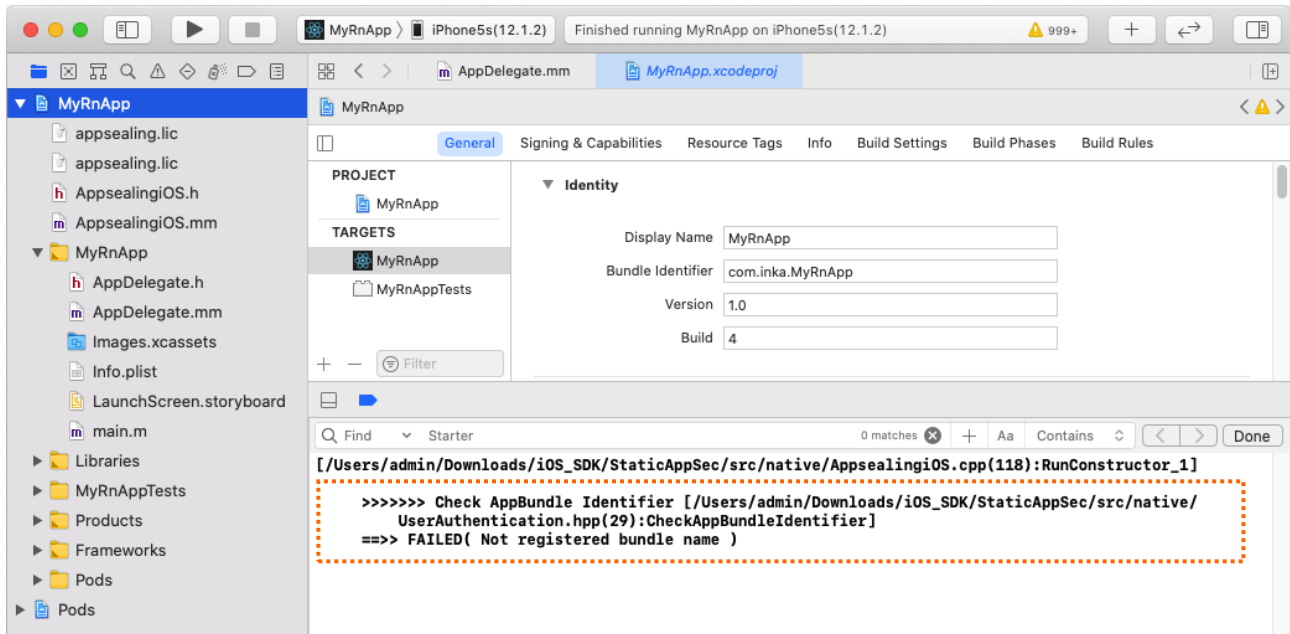
"AppSealingiOS.mm" 파일이 프로젝트에 포함되어 있지 않은 경우 "File > Add Files to "MyRnApp" ..." 메뉴를 실행하고 AppSealing SDK 경로에서 파일을 선택하여 프로젝트에 포함 시키십시오.



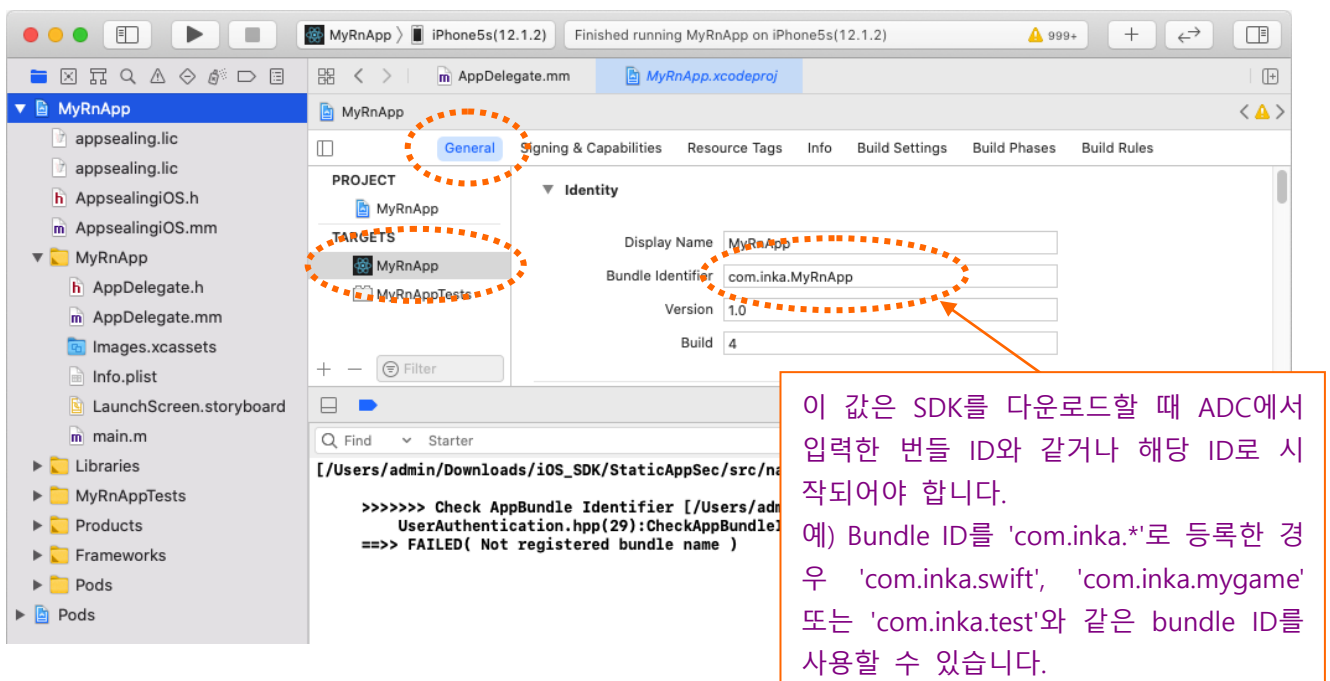


7-2 Execution error (I) 앱이 실행되고 곧 바로 종료됩니다.

기기에서 앱이 실행 직후 종료된 경우 bundle id가 유효한지 실행 로그 메시지를 확인하십시오.

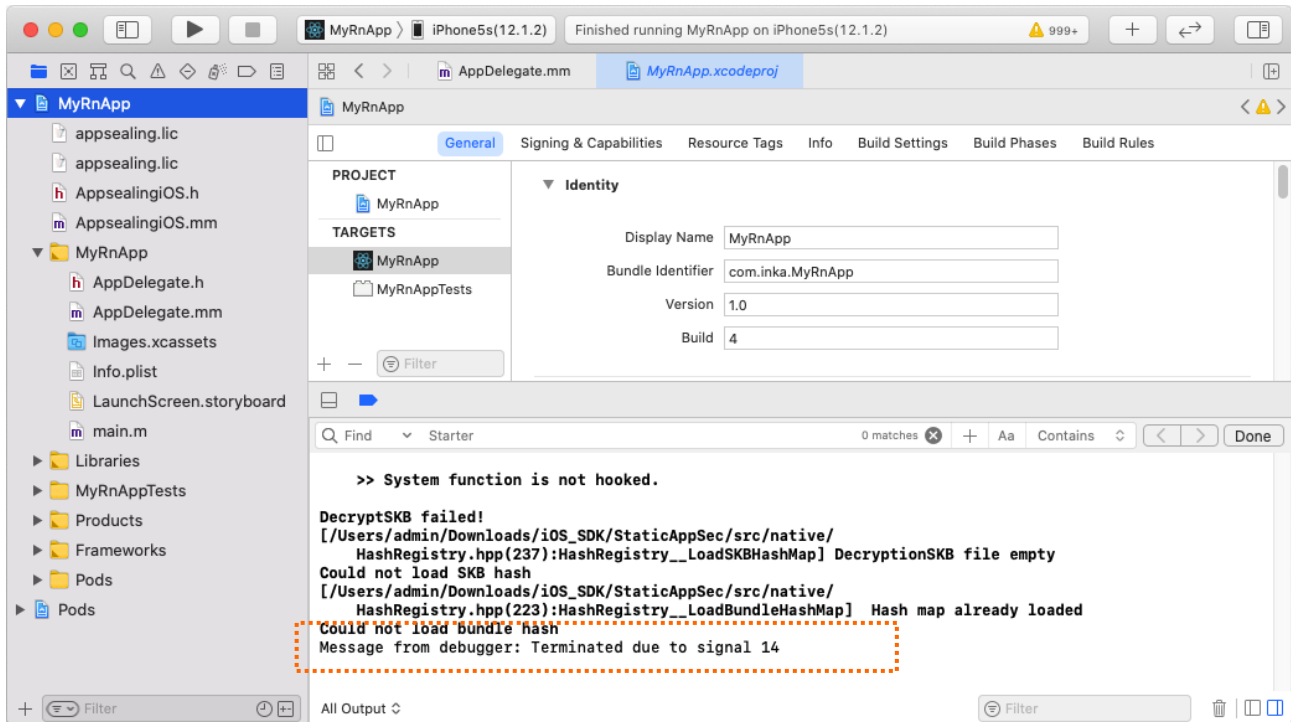


로그 메시지에 "====>>>> FAILED(Not registered bundle name)"와 같은 문자열이 포함되어 있으면 현재 프로젝트의 bundle id가 AppSealing SDK를 다운로드하는 동안 AppSealing Developer Center (ADC)에 제대로 등록되어 있지 않다는 의미입니다. ADC에서 SDK를 다운로드할 때 입력한 값과 같은 bundle id를 사용하십시오.



7-3 Execution error (II) 앱이 실행 중 갑자기 종료됩니다.

실행 후 약 20초 후에 갑자기 앱이 종료된 경우, 실행 로그를 확인하여 앱이 릴리스로 실행되었는지 확인할 수 있습니다.

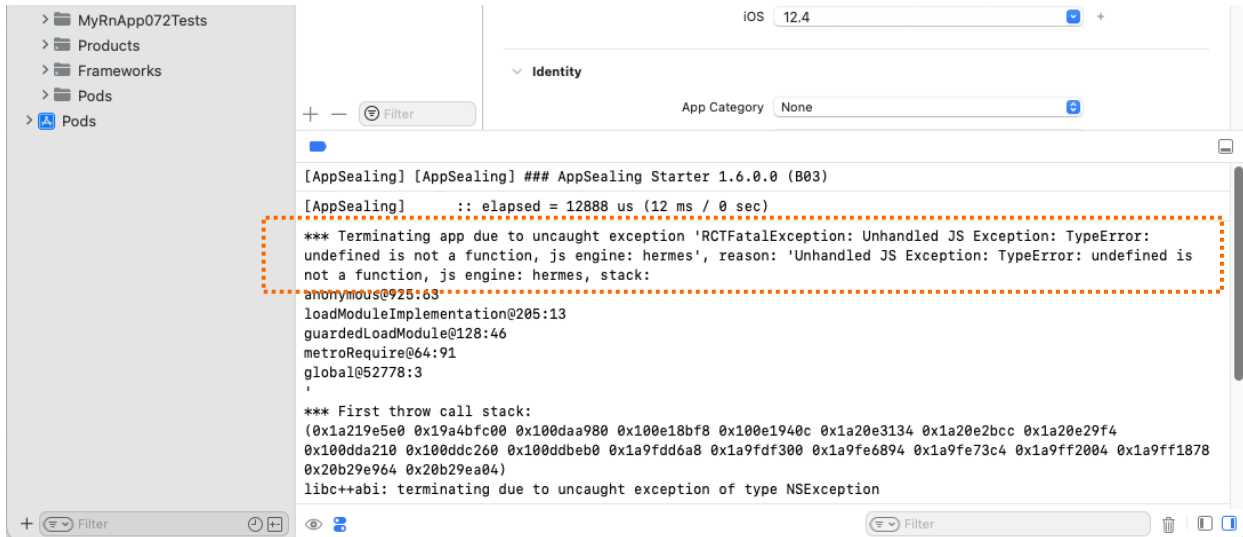


로그 메시지에 "Message from debugger: Terminated due to signal 14" 와 같은 문자열이 포함되어 있는 경우 앱이 release로 빌드, 실행되었고 AppSealing 보안 로직이 활성화되었음을 알 수 있습니다. Release 모드의 AppSealing 보안 로직은 탈옥된 장치, 실행 파일의 복호화 된 상태, 디버거 연결과 같은 보안 침입을 확인하며, 어떤 문제가 있을 경우 사용자의 조치나 기타 상황에 관계없이 20초 후에 앱을 닫습니다.

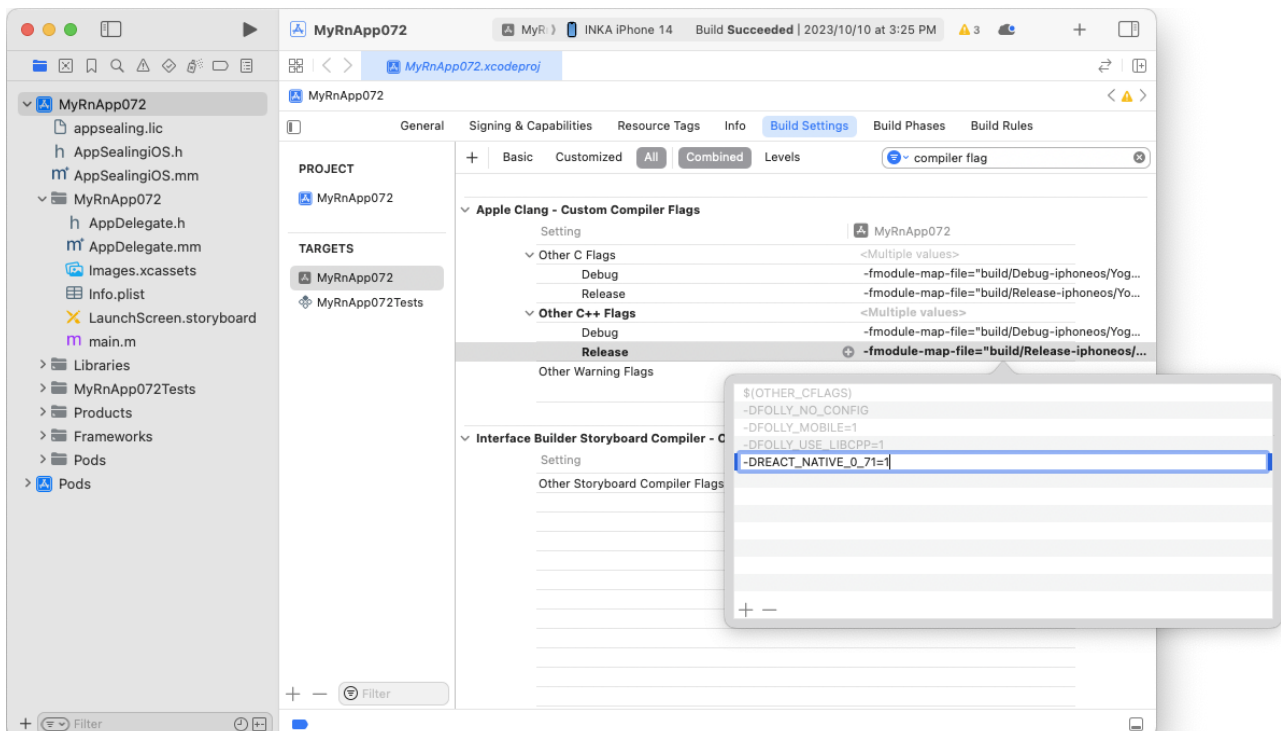
Release 모드에서는 AppSealing 로직이 활성화되고 앱이 종료되므로 AppStore 또는 TestFlight에 배포하기 전까지 디버그 모드로 앱을 테스트하십시오. AppSealing 로직은 release 모드로 빌드 시 Xcode 장치에서 앱을 실행하면 디버깅 환경으로 탐지하거나 실행 파일이 비정상적으로 해독되어 있음을 감지하고 20초 후에 앱을 종료합니다.

7-4 Execution error (III) 앱이 실행 즉시 종료됩니다.

실행 후 즉시 앱이 종료된 경우, 실행 로그를 확인하여 "Unhandled JS Exception: TypeError: undefined is not a function" 이라는 문장이 포함되어 있을 경우 빌드 설정에 C++ 컴파일 플래그가 제대로 설정되어 있는지 다시 확인해 보십시오. (3-2 참조)

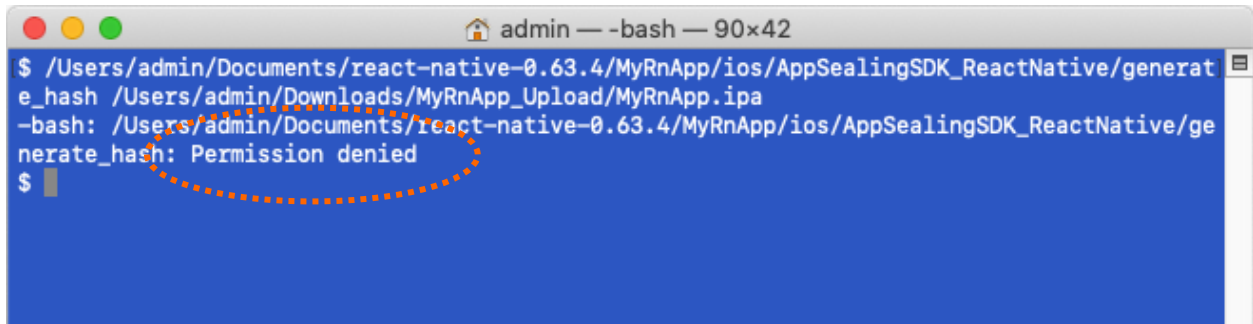


빌드 설정 – Apple Clang Custom Compiler Flags – Other C++ Flags – Release 설정 값에 "-DREACT_NATIVE_0_71=1" 값이 설정되어 있는지 확인하십시오.



7-5 무결성/인증서 스냅샷 생성 스크립트가 실행되지 않습니다.

3-5 단계의 무결성/인증서 스냅샷 정보 생성 단계에서 아래와 같이 스크립트 실행이 실패하고 "권한 없음" 오류 메시지가 출력되는 경우가 발생할 수 있습니다.

A terminal window titled 'admin — -bash — 90x42' showing a command execution. The command is `$ /Users/admin/Documents/react-native-0.63.4/MyRnApp/ios/AppSealingSDK_ReactNative/generate_hash /Users/admin/Downloads/MyRnApp_Upload/MyRnApp.ipa`. The output is `-bash: /Users/admin/Documents/react-native-0.63.4/MyRnApp/ios/AppSealingSDK_ReactNative/generate_hash: Permission denied`. The error message is circled in orange.

```
$ /Users/admin/Documents/react-native-0.63.4/MyRnApp/ios/AppSealingSDK_ReactNative/generate_hash /Users/admin/Downloads/MyRnApp_Upload/MyRnApp.ipa
-bash: /Users/admin/Documents/react-native-0.63.4/MyRnApp/ios/AppSealingSDK_ReactNative/generate_hash: Permission denied
$
```

이 경우 아래와 같이 스크립트 파일에 실행 권한을 추가하는 명령을 실행 하고 다시 시도해 보십시오.

```
$ chmod +x /MyRnApp/ios/AppSealingSDK_ReactNative/generate_hash
```